

基于风险分位规划与储能反馈的微网购电调控

摘 要

针对含光伏与储能的微网购电问题，本文以十分钟为步长，将供需平衡、储能损耗、购电结算和信息发布时间纳入规划，分别研究确定性日循环、历史预测下固定计划、日内调整及波动电价四种情形。其中，常规购电提前支付，紧急补购按五倍电价结算，储能库存又连接相邻时段。为协调三者，采用“历史预测、风险修正、线性规划、实际反馈”的计算流程，并使日末真实库存连续传入次日。

问题一以日购电费最小为目标，逐项建立能量、功率、库存与日循环约束。证明允许不利用余电时充放电互斥可作等价线性松弛，并用独立混合整数规划复核。最优日购电量为 59482.70 kWh，费用为 35126.95 元；相比同输入无储能方案，费用降低 26.90%。容量、功率及效率扰动表明，三类设备限制对电费的影响程度不同。可用容量增加 20% 可降费 3.58%，相同幅度提高功率仅降费 0.12%，说明该运行点的容量限制更突出。

问题二仅用已完成日期预测负荷和光伏，以历史残差分位修正净负荷，求解零时固定计划，再按真实需求执行储能与紧急补购。1 月选得风险分位 0.80，2—12 月总费为 1390.82 万元。提高分位可减少紧急电，但过高分位导致提前购电支出增加。正式期共 334 天，紧急电费占总费 4.43%；分位由 0.80 增至 0.90 后，总费反增约 23.22 万元。

问题三按已发布光伏预报更新剩余计划，采用历史与附件预报各占 0.50 的融合预测和 0.65 风险分位。按最终净调整收费，总费为 1331.58 万元，较问题二下降 4.26%。保持调整机会一致后，新增小时预报贡献 1.72 万元费用改善，18 点新预报未显示可辨认的收益；逐笔合约重优化后费用为 1346.05 万元。紧急电量下降 74.84%，但发生天数由 115 天增至 139 天，表明总量改善与事件频率变化需要分别评价。

问题四用历史预测价格规划、实际交付价格结算，两种决策频率的总费为 1467.58 万元、1404.49 万元，日内调整降费 4.30%。共同热启动下将预测价差幅度扰动 $\pm 20\%$ ，实际总费变化均小于 0.12%，策略排序保持不变；逐月选参和所测试的确定性价格感知反馈未改善累计费用。

各方案通过物理、信息顺序和费用核验。通过改变未来实测和未发布预报，检查既有决策不受未来信息影响；再由逐段记录独立重算电量与账单，核对论文表格与工作簿的一致性。本文仅对问题一给出确定性最优结论，后续方案为可行的预测与反馈策略；全部经济比较基于同年数据，尚不构成未见年度的泛化证据。

关键字：微网 储能调度 风险分位 因果预测 滚动评估

一、 问题重述

1.1 研究背景与任务

研究对象为含光伏和储能设备的微网。微网不向外部电网售电，日常用电由光伏、已经约定的外网购电及储能共同承担；电力不足时，可以按对应电价的五倍紧急购电。储能额定容量为 12000 kWh，允许储电量为 1200–10800 kWh，充放电最大功率均为 5000 kW。

题目要求解决四类逐步增加信息与决策自由度的问题：其一，在预测曲线确定且当天储电量闭合的条件下，求一天的最低购电费用；其二，仅以历史运行数据制定每日零时计划，并在真实负荷与光伏实现时调整储能；其三，结合每日零时、6 时、12 时和 18 时发布的光伏预报，允许对尚未交付的购电计划进行调整，分析新预报的价值；其四，将上述策略推广至波动电价。除全年结果外，还需给出指定时段与四个指定日期的详细调度表。

1.2 数据整理与时间约定

附件一包含 144 个时点的电价及典型负荷、光伏预测，附件二和附件四包含 2025 年 365 天、每天 144 个时点的实际功率和电价，附件三包含每日 4 次、每次未来 24 小时的光伏预报。日期连续，所需数值无缺失、负数或非有限值。输出评价区间为 2025 年 2 月 1 日至 12 月 31 日，共 334 天、48096 个十分钟区间；1 月用于历史初始化、候选参数验证和控制热启动。

输入记录 00:10 被解释为 00:00–00:10 区间的平均代表功率，因而单段电量等于该行功率乘以 $\Delta = 1/6$ 小时。这一解释与输入的右端时标一致；由于题目未区分瞬时采样与平均功率，另对问题一作梯形积分敏感性检验。原结果模板的标签整体向后偏移十分钟，故输出统一标为 00:00–00:10 至 23:50–24:00，保持数据次序不变。

附件三第 k 个预报值对应发布时间之后第 k 小时的目标，不能按当日第 k 小时读取。本文使用目标整点间线性插值；发布时间的插值起点取当时已经完成区间的光伏观测。新发布的 6 时预报最早影响 6:00–6:10，不能回写已经交付的区间。

附件一的典型负荷与附件二全年同刻均值几乎一致。如果把这一曲线作为问题二至四的预测先验，就会隐含使用未来统计量。因此后续问题仅引用附件一已给定的固定电价，不使用其典型负荷或光伏作为历史先验。问题一则依题意直接使用附件一的给定预测。

二、模型的假设

1. 每个十分钟区间内功率近似恒定，控制器能够响应当段已实现负荷和光伏。题目未给出额外响应延迟、输电限制、老化费用或自放电，本文不另行编造这些参数。
2. 充、放电效率分别取 0.9，因而往返效率为 0.81；储能的充放电功率在交流母线侧计量。将 90% 解释为往返效率的情形另作敏感性比较。
3. 不售电，外网购电没有题给的功率上限。允许不利用无法吸收的光伏或不接收已经付费的计划电量，其购电支出照付。两类余量统一称为未利用电量，不全部称为弃光。
4. 问题一执行日循环等式。问题二至四从 1 月 1 日 6000 kWh 连续运行；规划器使用日末预测储电量至少 6000 kWh 的库存恢复设计，实际执行不强制达到这一目标，也不在午夜免费重置。
5. 波动电价的未来值不视为提前公布。规划使用历史预测价，执行和结算使用对应交付段已实现的价格；主解释中的“交易时刻”取该交付区间。

三、符号说明

表 1 主要符号与单位

符号	含义	单位
d, t	日期索引与日内十分钟区间索引， $t = 0, \dots, 143$	—
L_t, V_t	当段实际负荷电量和光伏电量	kWh
g_t, q_t	零时原始购电计划与最终常规交付量	kWh
c_t, d_t^{bat}	母线侧充电输入量与放电输出量	kWh
S_t	区间开始时的实际储电量	kWh
e_t, w_t	紧急购电与未利用电量	kWh
$p_t, \hat{p}_t$	实际结算价与决策时预测价	元/kWh
$\hat{N}_t, \tilde{N}_t$	点预测净负荷与风险调整净负荷	kWh
λ, α	附件三预测权重与风险分位参数	—

为简化以下公式，放电量记为 $b_t = d_t^{\text{bat}}$ ，从而避免与日期索引混淆。

四、 问题分析

4.1 预测、购电计划与实时执行的分工

问题一是确定性资源配置问题，可以直接建立线性规划。问题二至四同时涉及预测和控制：即使某次规划在预测输入下最优，也不能据此推出有预测误差时的全年策略最优。因此本文将预测、常规购电规划和实时执行分为三个环节，分别约束信息的可用时刻、物理动作和费用核算。储能跨时间耦合又使预测误差的经济影响依赖库存及电价，故预测精度只能作为辅助指标，主要评价量应为真实结算费用。

4.2 四个问题的递进关系

1. 问题一的预测曲线已经给定，重点是把电量平衡、储能损耗和日循环边界写成可核验的优化模型，并确认线性松弛不会改变最优购电费用。
2. 问题二不能读取未来真实负荷和光伏，需要先构造历史预测，再用非对称购电代价确定风险修正，最后通过实时储能反馈与紧急购电实现可行运行。
3. 问题三同时增加预报信息和日内调整机会，必须区分这两种变化的经济作用，并明确多次申报是按最终净调整还是逐笔成交结算。
4. 问题四再加入未来价格未知这一因素，需要以因果价格预测形成计划、以实际交付价格结算，并保持与固定电价问题相同的物理约束和比较口径。

4.3 相关方法与本文定位

储能调控需要在不同电价和预测误差之间分配有限库存。分时电价下光伏储能经济调度及日前、日内协调优化已有相关研究^[1,2]。模型预测控制在每次获得新观测后，重新预测并优化未来时域，仅实施首个动作；确定性滚动控制与多情景鲁棒控制具有不同的不确定性处理方式^[3]。动态电价研究也强调预测驱动的可实施策略应与全知性能参考区分^[4]。

日前、日内协调还可结合多时间尺度 MPC 与分布鲁棒方法，显式处理场景分布不确定性^[5]。本文采用容易核验的风险分位规划与显式储能反馈，未实现该类分布鲁棒模型，也不把经验分位解释为联合概率约束或条件风险价值。通过历史预测基准、严格预报信息对照、替代交易合约以及价格感知反馈比较，检验结论对建模选择的依赖性。

五、模型的建立与求解

5.1 问题一：确定性购电规划

5.1.1 输入、假设与变量

附件一给出了同一天 144 个十分钟区间的负荷、光伏预测和电价。按照题意，本问将这些曲线视为确定输入；需要选择的是在哪些区间购电、充电和放电。分时电价下的储能经济调度已有相关研究^[1]，本题的特殊约束在于日初和日末库存必须一致，故不能逐时段独立选择购电量。

记区间集合为 $\mathcal{T} = \{0, \dots, 143\}$ ，以 $L_t = P_t^L \Delta$ 、 $V_t = P_t^V \Delta$ 表示负荷和光伏电量。每段设置购电量 g_t 、充电量 c_t 、放电量 b_t 、未利用电量 w_t 及段末储电量 S_{t+1} 。五类变量均为连续量，单位为 kWh；电价 p_t 的单位为元/kWh。当前储电量连接相邻时段，不能在各时段独立重置。

本问没有另设紧急购电变量，因为全部输入已知，可在零时一次性安排满足需求的购电量。效率按充、放电各 0.9 解释，允许免费不利用余电，其余物理条件采用题给参数。效率含义和可用容量对结果的影响，在本问末尾单独检验。

5.1.2 目标函数与分条约束

以全天外网购电支出最小为目标。光伏没有题给的使用费用，储能损耗通过能量递推体现，不再额外添加一项损耗电费：

$$\min C_1 = \sum_{t \in \mathcal{T}} p_t g_t. \quad (1)$$

目标函数中“电价乘电量”的结果为元；若直接将功率代入，费用会多算六倍。为使该目标对应可实际运行的日计划，依次建立以下六类约束。

a) 供需平衡。当段购电、光伏和储能放电应覆盖负荷与充电需求。引入未利用电量 w_t ，将题目中的“不低于”写成等式：

$$g_t + V_t + b_t = L_t + c_t + w_t, \quad t \in \mathcal{T}. \quad (2)$$

$w_t \geq 0$ 既可表示无法吸收的光伏，也可表示不接收的已付费外网电量。外网电费仍按 g_t 支付，因此不能再从目标函数中扣掉 $p_t w_t$ 。

b) 储电量递推。充电量和放电量均在交流母线侧计量，输入 1 kWh 只能增加 0.9 kWh 库存，输出 1 kWh 则需要消耗 1/0.9 kWh 库存：

$$S_{t+1} = S_t + \eta_c c_t - b_t / \eta_d, \quad \eta_c = \eta_d = 0.9. \quad (3)$$

144 个区间对应 145 个状态， S_t 为本段开始库存，不使用段末值。

c) 充放电功率限制。由于决策变量是电量，5000 kW 的功率上限必须先乘十分钟的时长：

$$0 \leq c_t, b_t \leq P_{\max} \Delta, \quad P_{\max} \Delta = 5000/6 \text{ kWh}. \quad (4)$$

即使储电量充足，单段放电仍不能超过 833.33 kWh；功率和容量分别约束放电速度与可持续时间。

d) 安全库存限制。在每个区间边界均需检查

$$1200 \leq S_t \leq 10800, \quad t = 0, \dots, 144. \quad (5)$$

额定容量为 12000 kWh，但实际可以调度的库存幅度为 9600 kWh，不能将额定容量全部用于套利。

e) 日循环边界。给定初始库存为 6000 kWh，问题一每天重复同一组输入，因此要求

$$S_0 = S_{144} = 6000. \quad (6)$$

仅给日末库存设置物理下限，会允许消耗期初储能来降低当日费用，不能代表可重复运行的日计划。

f) 购电方向及运行模式。微网不向外网售电，余量不能为负，同一段储能只允许一种运行方向：

$$g_t, w_t \geq 0, \quad c_t b_t = 0. \quad (7)$$

前两个不等式限制交易和余电的方向，乘积约束限制设备状态，不能用购电非负代替充放电互斥。

5.1.3 汇总模型与线性化

将目标和约束合并，得到问题一的规划模型：

$$\begin{aligned} (P_1) \quad & \min \sum_{t=0}^{143} p_t g_t, \\ \text{s.t.} \quad & g_t + V_t + b_t = L_t + c_t + w_t, \\ & S_{t+1} = S_t + 0.9c_t - b_t/0.9, \\ & 0 \leq c_t, b_t \leq 5000/6, \quad g_t, w_t \geq 0, \\ & 1200 \leq S_t \leq 10800, \quad S_0 = S_{144} = 6000, \\ & c_t b_t = 0 \quad (t \in \mathcal{T}). \end{aligned} \quad (8)$$

库存限制另包括日末 $t = 144$ 。只有互斥条件不是线性的。通常可引入二元变量 y_t ，以 $c_t \leq (5000/6)y_t$ 、 $b_t \leq (5000/6)(1 - y_t)$ 代替乘积约束，得到混合整数线性规划。但在本题余电处置条件下，可以先求解更简洁的连续线性规划。

考虑松弛模型中某一同时充放电的时段，令

$$\delta = \min\{c_t, b_t/(\eta_c\eta_d)\}, \quad c'_t = c_t - \delta, \quad b'_t = b_t - \eta_c\eta_d\delta. \quad (9)$$

再令 $w'_t = w_t + (1 - \eta_c\eta_d)\delta$ 。该变换使 c'_t 、 b'_t 中至少一个为 0，并保持

$$\eta_c c'_t - b'_t/\eta_d = \eta_c c_t - b_t/\eta_d. \quad (10)$$

故全部库存不变，充放电功率不增加；增加的未利用量恰好补回能量平衡的差额。购电量 g_t 没有改变，目标费用保持不变。因此，线性松弛的任一最优解都能转成同费用的互斥解，松弛不会降低实际可达的最低电费。

这一结论依赖允许不利用余电，且目标不奖励额外充放电。若以后加入弃电惩罚、禁止弃电或循环补贴，应重新检查等价关系，不能直接沿用本次线性化。

5.1.4 求解步骤与最优性核验

去掉互斥乘积后，使用 $\mathbf{x} = (\mathbf{g}, \mathbf{c}, \mathbf{b}, \mathbf{S}_{1:144}, \mathbf{w})$ 作为求解向量，共 720 个连续变量。每个时段有一条平衡等式和一条状态等式，共 288 条；容量、功率和日末等式通过变量边界给定。系数矩阵只连接本段动作与相邻库存，适合使用稀疏矩阵存储。

计算通过 SciPy 的 HiGHS 接口完成^[6]；所用求解器包含对偶修正单纯形等线性优化方法^[7]。求解过程为：

1. 将 144 行功率乘以 1/6，形成负荷、光伏电量与目标系数向量，检查单位和时段顺序。
2. 按两类等式逐行装配系数矩阵，设置充放电、库存及首末库存边界。
3. 求解纯电费 LP，保存最低费用；另以微小选解项 $10^{-6} \sum_t (c_t + b_t) + 10^{-7} \sum_t w_t$ 确定交付用的退化解。
4. 将选解后的实际电费与纯电费下界比较，不把选解项计入购电账单。
5. 独立建立含 144 个模式变量的 MILP，检查目标差、整数性和物理残差。
6. 将通过核验的十分钟结果按指定区间聚合，生成购电表、储能表和完整工作簿。

纯电费 LP 和独立 MILP 均得到 35126.948589 元，MILP 最优性间隙为 0，最大约束残差约 7.62×10^{-12} kWh。加入选解项的交付调度也达到相同纯电费。因此，问题一的最优性有目标下界和独立整数模型的共同支持；该证据不延伸到后续含预测误差的全年策略。

5.1.5 购电结果与储能作用

最优日购电量为 59,482.70 kWh，全天费用为 35,126.95 元。表2按题面表 1 给出六个十分钟区间和全天购电结果；表3按题面表 2 列出六个四小时区间的充放电量以及首末库存。两表电量单位均为 kWh，购电费单位为元；充放电区间接题面顺序从左至右、从

上至下排列。

表 2 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天购电量		59,482.70		全天购电费	
				35,126.95	

表 3 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,500.00	0.00	4:00–8:00	833.33	6,365.84
8:00–12:00	4,787.96	1,703.00	12:00–16:00	5,286.04	91.10
16:00–20:00	0.00	5,780.13	20:00–24:00	5,333.33	2,859.87
0:00 储电量		6,000.00	24:00 储电量		6,000.00

为解释购电时段的安排，图1同时给出购电、充放电、库存和电价。库存上限使低价区间不能无限购电，功率上限使高价区间不能一次性释放全部库存，日循环条件又要求当天用掉的库存得到补充。

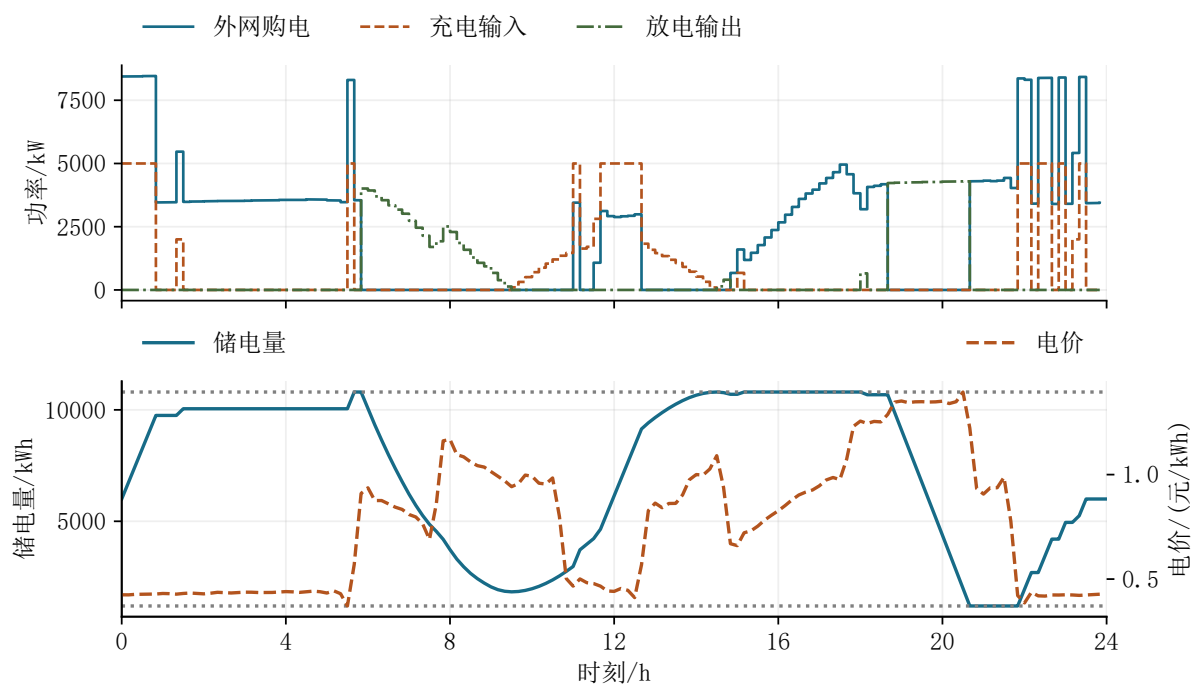


图 1 问题一的购电、储能及电价关系

在相同已知曲线下，关闭储能并逐段购买正净负荷，费用为 48,052.05 元。储能使

当日电费减少 12,925.10 元，降幅 26.90%。这是同一确定输入和同一费用规则下的比较，节省主要来自光伏余量利用与跨时段购电转移。

全天充电输入为 20,740.67 kWh，放电输出为 16,799.94 kWh，两者相差 3,940.73 kWh。由日循环边界对式(3)求和可得

$$\sum_t b_t = \eta_c \eta_d \sum_t c_t = 0.81 \sum_t c_t. \quad (11)$$

该差额正是充放电往返损耗，不能把它视为缺货。若只考虑从低价段向高价段转移 1 kWh 电量，只有高价超过低价除以 0.81 时才有套利空间；实际调度还受到同时到达的光伏、库存和功率限制。四小时汇总中充电和放电同时为正，只说明该区间内不同十分钟段执行了不同动作。

5.1.6 容量、功率与效率灵敏度

固定附件一曲线和 6000 kWh 首末库存，分别改变可用库存幅度 B_u 、充放电功率上限 $P_{\max}$ 和单向效率 η ，每次只改变一个参数并重新优化。容量实验令库存范围为 $[6000 - B_u/2, 6000 + B_u/2]$ ，因此改变的是可用窗口，而不是额外赠送初始电能。各扰动的数值明细见附录B.1表55，费用变化以主方案为基准。可用容量增加 20% 使费用下降 3.58%，同幅度提高功率只降费 0.12%，说明该运行点的库存限制更突出。单向效率由 0.90 提高到 0.95 又可降费 3.87%，对应充电损耗减少。各扰动解通过能量平衡、首末库存、功率、互斥和费用核验，LP 目标与对偶下界一致。因未计设备投资，不据此推算扩容回收期。

若 90% 指往返效率，两侧取 $\sqrt{0.9}$ ，费用为 33,801.50 元；采用梯形积分时为 35,042.54 元。二者仅作解释对照，主结果仍采用区间均值和双侧 0.9 效率。

5.2 问题二：固定计划与实时反馈

5.2.1 信息条件与决策变量

本问电价已知，但当天负荷和光伏尚未实现。零时购电量一经确定就需要支付，未使用也不退款；供电缺口按五倍电价补购。因此，制定计划需要权衡提前多购与事后补购的不同代价。

记 $\mathcal{H}_d$ 为第 d 天零时的历史观测， $\hat{N}_{d,t}$ 为净负荷点预测。规划变量为固定计划 $g_{d,t}$ 及预测充放电 $\hat{c}_{d,t}, \hat{b}_{d,t}$ 、预测库存 $\hat{S}_{d,t}$ 、预测余量 $\hat{w}_{d,t}$ 。真实动作与状态用 c, b, S, e, w 记录，带帽变量仅用于制定计划。

沿用问题一的效率、母线侧计量、容量和功率约束；改变的是信息条件与日末要求。真实库存从 1 月 1 日 6000 kWh 起连续承接，预测规划设置日末恢复目标 $B = 6000$ kWh，真实执行只受物理边界约束。该目标用于减轻逐日规划的短视，不是题目要求每天实际回到 6000 kWh。

5.2.2 目标函数与风险净负荷

评价对象是正式期真实现金支出：

$$C_2 = \sum_{d=31}^{364} \sum_{t=0}^{143} (p_t g_{d,t} + 5p_t e_{d,t}). \quad (12)$$

日期索引从1月1日 $d = 0$ 开始。为处理零时未知的 $e_{d,t}$ ，本文由历史残差修正净负荷，在风险输入下最小化当日常规电费，再按真实需求执行反馈。这是对随机购电问题的规划近似。

先说明点预测的构造。有一周以上历史时，利用此前最多四个同星期日的同刻负荷：

$$\hat{P}_{d,t}^L = \frac{\sum_{k=1}^{K_d} 0.55^{k-1} P_{d-7k,t}^L}{\sum_{k=1}^{K_d} 0.55^{k-1}}, \quad K_d = \min\{4, \lfloor d/7 \rfloor\}. \quad (13)$$

历史不足7天时改用此前最多7天，以 0.8^{k-1} 衰减加权。光伏采用较短的历史窗口：

$$\hat{P}_{d,t}^{V,H} = \frac{\sum_{k=1}^{\min(7,d)} 0.75^{k-1} P_{d-k,t}^V}{\sum_{k=1}^{\min(7,d)} 0.75^{k-1}}, \quad d \geq 1. \quad (14)$$

首日没有历史，负荷取4500kW、光伏取0，只用于初始化。两类功率预测相减后乘 Δ ，得到 $\hat{N}_{d,t}$ 。附件一典型负荷几乎等于全年同刻均值，故不将其作为本问先验，否则零时预测会间接含有未来样本。

对每个已完成日期 j ，保存当时预测与实际净负荷的残差 $R_{j,t} = N_{j,t} - \hat{N}_{j,t}$ 。在 $\max(7, d-28) \leq j < d$ 内，将同刻及前后各3段残差合并，记为 $\mathcal{R}_{d,t}$ ；区间端点以最近端点复制补齐。规划输入为

$$\tilde{N}_{d,t} = \hat{N}_{d,t} + Q_\alpha(\mathcal{R}_{d,t}). \quad (15)$$

分位数采用排序样本间线性插值，无可用历史残差时不添加修正。较大的 α 通常使提前备电更多，但可能增加已经付费的未利用电量。该经验分位只修正各段规划输入，既没有构造联合机会约束，也不表示系统具有 α 的供电可靠率。

由此，零时实际求解的目标为

$$\min \hat{C}_{2,d} = \sum_{t=0}^{143} p_t g_{d,t}. \quad (16)$$

它与式(12)作用不同：前者是在风险输入下选择计划的优化目标，后者是在实际路径上比较策略的评价指标。分位参数应按后者的历史费用选择，不能仅按预测误差或规划目标选择。

5.2.3 约束条件与汇总模型

a) 预测供需平衡。以 $\tilde{N}_{d,t}$ 替代未知的实际净负荷，要求

$$g_{d,t} + \hat{b}_{d,t} = \tilde{N}_{d,t} + \hat{c}_{d,t} + \hat{w}_{d,t}. \quad (17)$$

本阶段不设置免费的缺口变量；当预测供电不足时必须通过 g 补足。实际紧急电留到交付阶段按五倍电价计费。

b) 储能递推与边界。预测态使用式(3)的同一效率关系，并满足 $0 \leq \hat{c}, \hat{b} \leq 5000/6$ 、 $1200 \leq \hat{S} \leq 10800$ 。允许未利用余量，故可采用问题一证明的线性松弛。

c) 起止库存。以真实到达的日初库存作为规划起点，而不是每日重新初始化：

$$\hat{S}_{d,0} = S_{d,0}, \quad \hat{S}_{d,144} \geq B, \quad B = 6000. \quad (18)$$

预测下保留库存，可以减轻逐日规划用尽储能的问题。已有两阶段调度研究也将日前库存边界传入日内决策^[8]，但本文的 6000 kWh 是自行设置的恢复目标，并非由该研究给出的最优参数。

d) 固定计划和可用信息。全天 $q_{d,t} = g_{d,t}$ ，交付前不能通过常规渠道追加或撤销； g_d 只能是 $\mathcal{H}_d$ 及真实日初库存的函数。用当天真值修正零时计划，会改变本问题的信息条件。

将上述关系合并，省略日期下标后得到

$$\begin{aligned} (\text{P}_2) \quad & \min \sum_{t=0}^{143} p_t g_t, \\ \text{s.t.} \quad & g_t + \hat{b}_t = \tilde{N}_t + \hat{c}_t + \hat{w}_t, \\ & \hat{S}_{t+1} = \hat{S}_t + 0.9\hat{c}_t - \hat{b}_t/0.9, \\ & 0 \leq \hat{c}_t, \hat{b}_t \leq 5000/6, \quad g_t, \hat{w}_t \geq 0, \\ & 1200 \leq \hat{S}_t \leq 10800, \\ & \hat{S}_0 = S_{d,0}, \quad \hat{S}_{144} \geq 6000. \end{aligned} \quad (19)$$

所有区间约束作用于 $t = 0, \dots, 143$ ，库存边界包括末端。得到计划后，预测库存不再覆盖真实状态，次日规划继续从实际运行结果开始。

5.2.4 参数选择与执行算法

候选分位为 $\{0.50, 0.65, 0.70, 0.80, 0.90, 0.95\}$ 。每个候选从 1 月 1 日 6000 kWh 起独立回放，仅以 1 月 15—31 日真实费用评分，选中 $\alpha = 0.80$ 。正式策略的 1 月运行预先固定使用 0.80，2 月起仍使用该参数；历史窗口随日期滚动，但不按正式期费用重新选参。

当段实际负荷和光伏到达后，定义常规电力相对净需求的余量 $z_t = g_t - (L_t - V_t)$ 。

若 $z_t \geq 0$ ，尽可能将余量存入电池：

$$\begin{aligned} c_t &= \min\{z_t, 5000/6, (10800 - S_t)/0.9\}, \\ b_t &= e_t = 0, \quad w_t = z_t - c_t. \end{aligned} \quad (20)$$

若 $z_t < 0$ ，先在功率与库存允许范围内放电，其余缺口紧急购入：

$$\begin{aligned} b_t &= \min\{-z_t, 5000/6, 0.9(S_t - 1200)\}, \\ c_t &= w_t = 0, \quad e_t = -z_t - b_t. \end{aligned} \quad (21)$$

两个最小值同时限制当段余缺、功率和可用库存，再按式(3)更新状态，保证平衡与安全边界。题目未限制紧急购电容量，因此剩余缺口均能补足。

完整的日循环算法如下：

1. 在零时读取前一日末库存和已完成日期观测，计算当日点预测。
2. 按固定 α 提取历史残差分位，构造 144 段风险净负荷。
3. 求解 (P_2) ，锁定全天 g_t ；保留预测态仅供诊断。
4. 在每个交付区间接收实际净负荷，按式(20)或(21)执行，并更新真实库存。
5. 对原计划和实际紧急电分别计费，检查当段供需平衡；日末库存直接传入次日。
6. 对 2—12 月费用求和，按题目日期导出计划、储能汇总和紧急事件。

该反馈优先补足当前缺口，未比较未来紧急电价；后文以价格感知反馈检验这一选择。

5.2.5 全年与指定日期结果分析

正式期 334 天总费用为 13,908,175.95 元，其中紧急电费 616,550.23 元，约占总费的 4.43%。表4给出费用和电量分解，表5汇总四个指定日期。正文按题面表 1、表 2 展示 9 月 23 日，其他日期及四日期紧急事件、账单见附录A.1，与 result2.xlsx 对应。

表 4 问题二的累计结果（2025 年 2—12 月）

指标	数值
原计划费/元	13,291,625.72
紧急费/元	616,550.23
总费用/元	13,908,175.95
常规购电/kWh	21,712,548.31
紧急购电/kWh	102,787.48

表 5 问题二的指定日期结果（2025 年）

日期	常规购电/kWh	紧急购电/kWh	全天总费用/元
03-20	68,386.72	0.00	41,537.79
06-21	37,707.19	0.00	22,414.49
09-23	68,094.10	892.40	47,814.59
12-21	99,788.74	0.00	64,446.61

各日期购电表中的购电量为零时确定的常规计划，“全天购电费”为该常规计划费用；紧急购电另列，含紧急费的总支出见全天账单。电量单位均为 kWh，费用单位为元。储能表按六个四小时区间求和，并列当日 0:00 与 24:00 的真实库存。表内保留两位小数，计算使用完整精度，逐项舍入后可能产生 0.01 的加总差。

附录紧急购电表将大于 10^{-7} kWh 的连续十分钟段合并为事件，终点为最后一段结束时刻；“无、0.00”表示当天没有紧急购电，后续空白表示没有更多事件。24:00 与 0:00+1 含义相同。后续各问沿用上述口径。2025.9.23 的购电和储能结果分别见表6和表7。

表 6 微网在指定时间段的购电量及全天的购电量和购电费（2025.9.23）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	499.87	14:00–14:10	0.00
16:00–16:10	531.67	18:00–18:10	775.50	20:00–20:10	0.00
全天购电量	68,094.10	全天购电费	42,079.63		

表 7 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.9.23）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,635.38	107.23	4:00–8:00	1,019.68	6,091.91
8:00–12:00	5,459.91	1,896.62	12:00–16:00	4,468.06	602.55
16:00–20:00	79.59	5,891.78	20:00–24:00	5,549.16	2,391.13
0:00 储电量	5,971.67	24:00 储电量	6,194.24		

四个指定日期中，9 月 23 日出现 892.40 kWh 紧急购电，其余三个日期没有。该日总费 47814.59 元，其中 5734.95 元来自紧急购电。图2选取这一日期展示风险计划、实际净需求及库存的关系，便于检查缺口发生的位置。

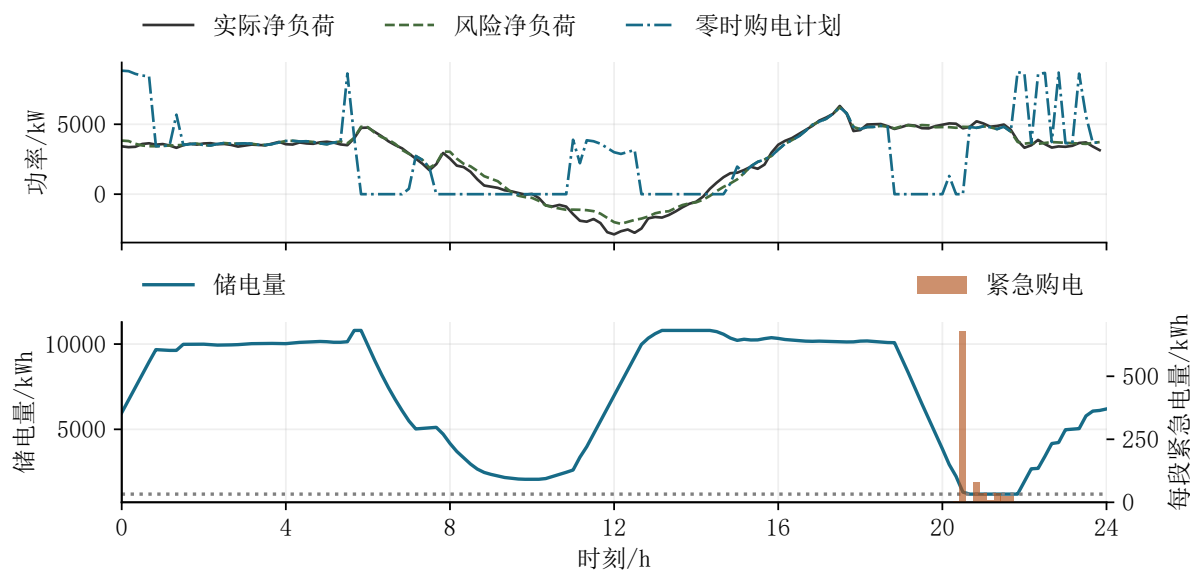


图 2 问题二在 9 月 23 日的固定计划与实际执行

常规富余电先存入电池，缺口先由电池补给，超出允许放电量后才使用紧急电。因此，同样的净负荷低估，在库存充足的低价段和接近放电边界的高价段会形成不同费用。

全年未利用电量 2,646,069.43 kWh 包含未消纳的付费常规电，不能全部称作弃光。全年 115 天发生紧急购电；期初、期末库存为 6,840.33 kWh、6,258.37 kWh，按实际现金计费，不对库存另作估值。

5.2.6 风险参数灵敏度检验

固定预测方法、反馈规则和共同 1 月热启动，将正式期分位改为 0.50、0.70 和 0.90，与主参数 0.80 比较。表8同时列出总费、紧急电量及年末库存，避免只以紧急电量下降判断参数改进。

表 8 问题二的分位参数扰动（2—12 月）

α	总费用/元	紧急购电/kWh	年末储电量/kWh
0.5	15,393,484.98	526,317.07	5,890.36
0.7	14,015,177.03	190,439.93	6,060.07
0.8	13,908,175.95	102,787.48	6,258.37
0.9	14,140,389.51	40,231.09	6,507.64

分位从 0.50 提高至 0.80，提前备电减少了高价补购，总费下降；继续提高至 0.90，紧急电量虽进一步下降，总费却增加约 23.22 万元，说明常规备电成本已超过新增的补购节省。主参数在所列候选中较省，但不意味着连续参数最优；各方案的年末库存不同，结论限于表中的现金路径。后续增加调整机会时，应重新按历史费用校准。

5.3 问题三：日内调整购电规划

5.3.1 新增信息与交易变量

问题三在固定电价下增加了两项条件：每天 00、06、12、18 点可接收未来整点光伏预报，且允许在后三个时刻修订尚未交付的购电量。新预报能够改变对未来光伏的判断，调整机会则允许根据实际库存和近期负荷重新安排购电。这两种作用必须分别比较，不能将总费用下降全部归因于预报精度。

记零时计划为 $g_{d,t}$ ，发布时间对应的区间索引为 $s \in \{0, 36, 72, 108\}$ ，在 s 修订的剩余计划记为 $q_{d,t}^{(s)}$ ，实际交付的最终常规电量记为 $q_{d,t}$ 。规划仍使用预测充放电和库存，实际执行仍采用问题二反馈。 $\mathcal{H}_{d,s}$ 包含此前日期的历史、当日 s 之前已完成区间观测，以及截至 s 已发布的预报。

本问沿用余电可以不利用、原计划款不返还的假设。主合约按同一交付段相对于零时原计划的最终净调整收费；未交付的追加申报允许被后续修订撤回。这是对多次调整累计方式的明确解释，逐笔成交不可撤回的替代制度另作检验。

5.3.2 费用目标与预报融合

总费用由原计划、调整和紧急购电三项组成：

$$C_3 = \sum_{d,t} [p_t g_{d,t} + 1.5p_t (q_{d,t} - g_{d,t})_+ + 0.5p_t (g_{d,t} - q_{d,t})_+ + 5p_t e_{d,t}]. \quad (22)$$

其中 $(x)_+ = \max(x, 0)$ ，求和覆盖正式期。增购 1 kWh 额外支付 $1.5p_t$ ，减购 1 kWh 仍保留原计划款并增加 $0.5p_t$ 违约金。不能将减购解释成退回原价后只损失 50%，否则会得到不同模型。

在零时仍求解 (P_2) 的计划电费目标，只将净负荷预测换成已发布预报与历史曲线的融合值。在日内 $s > 0$ ，原计划费用已确定，以剩余时域的增购费用最小为目标：

$$\min \hat{C}_{3,d,s} = \sum_{t=s}^{143} 1.5p_t (q_{d,t}^{(s)} - g_{d,t}). \quad (23)$$

本式采用 $q_{d,t}^{(s)} \geq g_{d,t}$ 的约束，其合理性将在下节说明。未来紧急支出仍通过风险净负荷间接防范，故该目标是日内规划近似，不等于直接优化随机全年账单。

附件三预报必须按绝对目标时刻对齐：发布于 h 点的第 k 列对应 $h + k$ 点。接收新行后，以最近一个已完成区间的光伏观测作为当前锚点，连接未来整点值，线性插值为十分钟预测 $\hat{P}^{v,F}$ ，到当日 24 点截断。不能把不同发布行的同列数据误当作同一时刻。

采用两来源线性融合：

$$\hat{P}_{d,t}^V = (1 - \lambda)\hat{P}_{d,t}^{V,H} + \lambda\hat{P}_{d,t}^{V,F}, \quad 0 \leq \lambda \leq 1. \quad (24)$$

预测组合可以利用不同来源的误差差异^[9]；光伏预测也有更复杂的自适应组合方法^[10]。本题采用可核对的凸组合，并通过真实购电费用选择权重，避免把预测精度较好直接等同于经济表现较好。

为避免近期观测使预测水平发生过大跳变，先定义区间截断函数：

$$\text{clip}(x, a, b) = \min\{b, \max\{a, x\}\}, \quad a \leq b. \quad (25)$$

其中 x 为待修正的数值， a 、 b 分别为下限和上限：小于 a 时返回 a ，大于 b 时返回 b ，在区间内则保留 x 。本文将它作用于无量纲修正比例，例如 $\text{clip}(1.40, 0.75, 1.25) = 1.25$ 。

在日内发布时刻 $s \in \{36, 72, 108\}$ ，再用此前 18 个已完成区间，即最近 3 小时的观测修正负荷水平：

$$\hat{P}_{d,t}^{L,(s)} = \hat{P}_{d,t}^L \text{clip}\left(\frac{\overline{P_{d,s-18:s}^L}}{\max(\hat{P}_{d,s-18:s}^L, 1)}, 0.75, 1.25\right), \quad t \geq s. \quad (26)$$

横线表示索引 $j = s - 18, \dots, s - 1$ 上的算术均值，分母中的 1 以 kW 计，用于避免预测均值过小导致比值异常。比值反映近期实际负荷相对于原预测的水平偏差；按式(25)将其限制在 $[0.75, 1.25]$ ，使每次修正幅度不超过原预测的 25%。该上下限是本文的控制设定，不是统计置信区间。以修正后的负荷和融合光伏形成点预测，再按式(15)加入该发布时刻对应的历史残差分位。每种 λ 及预报信息集合分别构造残差池，且只读取此前日期。

5.3.3 分条约束与汇总模型

a) 剩余时域供需平衡。仅对 $t \geq s$ 规划未来交付：

$$q_{d,t}^{(s)} + \hat{b}_{d,t} = \tilde{N}_{d,t}^{(s)} + \hat{c}_{d,t} + \hat{w}_{d,t}. \quad (27)$$

已交付区间不重新优化，其实际费用和库存变化均保留。

b) 真实库存起点。本次规划从 $\hat{S}_{d,s} = S_{d,s}$ 开始，并要求 $\hat{S}_{d,144} \geq 6000$ ；不能沿用零时预测的 $\hat{S}_{d,s}$ 代替实际库存。所有预测态继续满足问题一的状态递推、功率和库存边界。

c) 原计划下界。若某段 $q < g$ ，将 q 提高到 g 并将差额计入未利用量，可以保持充放电和库存不变，却免去减购违约费。因此在主假设下不减购原计划具有弱优势，可以加入

$$q_{d,t}^{(s)} \geq g_{d,t}, \quad t \geq s. \quad (28)$$

这个下界是零时 g_t ，不是此前的 $q_t^{(s-36)}$ 。后续修订可以撤回尚未交付的追加量，仍只在交付时按最终净差收费。

d) 信息与动作时点。本次 $q^{(s)}$ 仅由 $\mathcal{H}_{d,s}$ 产生；交付段实际需求到达后，只执行充放电反馈和紧急补购。当段真值不用于回写零时计划或此前已经交付的量。

省略日期下标，得到每次日内重规划的完整形式：

$$\begin{aligned}
(\mathbf{P}_3^{(s)}) \quad & \min \quad \sum_{t=s}^{143} 1.5p_t(q_t^{(s)} - g_t), \\
\text{s.t.} \quad & q_t^{(s)} + \hat{b}_t = \tilde{N}_t^{(s)} + \hat{c}_t + \hat{w}_t, \\
& \hat{S}_{t+1} = \hat{S}_t + 0.9\hat{c}_t - \hat{b}_t/0.9, \\
& 0 \leq \hat{c}_t, \hat{b}_t \leq 5000/6, \quad \hat{w}_t \geq 0, \quad q_t^{(s)} \geq g_t, \\
& 1200 \leq \hat{S}_t \leq 10800, \quad \hat{S}_s = S_{d,s}, \quad \hat{S}_{144} \geq 6000.
\end{aligned} \tag{29}$$

区间约束作用于 $t = s, \dots, 143$ ，库存边界还包括 144。剩余时域分别为 108、72、36 段，相应线性规划为 540、360、180 个连续变量。充放电互斥仍可按问题一的论证处理。

5.3.4 求解与日内执行步骤

权重候选为 $\{0, 0.25, 0.50, 0.75, 1\}$ ，与六个分位组成 30 个候选。只按 1 月 15—31 日真实费用选参，得到 $(\lambda, \alpha) = (0.50, 0.65)$ 。正式策略的 1 月热启动固定采用纯附件三、0.80 分位和三次日内调整，2 月 1 日才切换到所选参数。候选评分回放与这条热启动路径分别保存，不能把月底选择结果倒灌到此前执行。

日前与日内协调优化已有相关研究^[2]。本问采用如下分时执行步骤：

1. 零时继承真实库存，读取已发布光伏预报，与历史曲线融合并作风险修正，求解全天原计划 g 。
2. 每到 06、12、18 点，接收本次新预报和此前已完成区间的负荷、光伏观测，更新剩余预测。
3. 以当前实际库存为起点，求解 $(\mathbf{P}_3^{(s)})$ ，仅覆盖尚未交付的常规计划；保存每次修订以供核账。
4. 在每个交付段以当前 q_t 替换问题二反馈式中的 g_t ，执行储能和必要的紧急购电。
5. 按最终 $q_t - g_t$ 而非每次申报变化累计主合约调整费，日末将实际库存传至次日。

日内 LP 只决定常规购电承诺，后续真实负荷仍可能偏离风险输入，故不能将该 LP 最优性描述为日内随机控制的全局最优性。

5.3.5 调度结果与费用来源

正式期总费用为 13,315,835.99 元，比问题二减少 592,339.96 元，降幅 4.26%。表9将费用、电量和库存放在同一口径下比较。

表 9 固定电价下两种策略的结果（2—12 月）

指标	零时固定计划	日内调整计划
原计划费/元	13,291,625.72	12,649,467.17
调整费/元	0.00	527,937.93
紧急费/元	616,550.23	138,430.89
总费用/元	13,908,175.95	13,315,835.99
常规购电/kWh	21,712,548.31	21,197,074.02
紧急购电/kWh	102,787.48	25,862.57
未利用电量/kWh	2,646,069.43	2,021,445.81
发生紧急购电的天数	115	139
期初储电量/kWh	6,840.33	7,089.87
期末储电量/kWh	6,258.37	5,987.72

原计划费由 13291625.72 元降至 12649467.17 元，紧急费由 616550.23 元降至 138430.89 元，两项节省抵消了 527937.93 元调整费。常规购电量和未利用电量也下降，说明本次收益并非只把紧急电重新命名为常规电，而是购电时点和数量共同发生了变化。

紧急购电总量下降约 74.84%，但发生紧急购电的天数从 115 天增至 139 天。由此应将结论表述为紧急电量及支出下降，不能写成紧急事件全面减少。两种主策略的热启动不同，首末库存已在表中报告；上述差额是完整策略路径的现金比较。

表10汇总四个指定日期，正文按题面格式列出 9 月 23 日，其他日期及紧急事件、账单见附录A.2，与 result3.xlsx 对应。购电表用相同的六列结构分别展示零时原计划和最终常规购电，两者之差才是净调整量。原计划部分的“全天购电费”为原计划费，最终部分为原计划费加调整费，均不含紧急费；储能表为实际执行值。

表 10 问题三的指定日期结果（2025 年）

日期	常规购电/kWh	紧急购电/kWh	全天总费用/元
03-20	67,646.32	0.00	41,647.19
06-21	35,854.93	0.00	20,909.49
09-23	67,230.48	170.85	43,904.67
12-21	96,803.32	0.00	62,129.52

2025.9.23 的购电和储能结果分别见表11和表12。

表 11 微网在指定时间段的购电量及全天的购电量和购电费（2025.9.23）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	401.83	14:00–14:10	0.00
16:00–16:10	520.87	18:00–18:10	763.88	20:00–20:10	0.00
全天购电量		65,285.63	全天购电费		40,305.24
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	401.83	14:00–14:10	0.00
16:00–16:10	520.87	18:00–18:10	793.71	20:00–20:10	0.00
全天购电量		67,230.48	全天购电费		43,084.92

表 12 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.9.23）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,254.93	310.26	4:00–8:00	1,030.38	6,166.49
8:00–12:00	5,281.76	1,896.62	12:00–16:00	5,321.25	534.56
16:00–20:00	111.95	5,842.00	20:00–24:00	5,560.36	2,696.82
0:00 储电量		6,177.67	24:00 储电量		6,196.96

9 月 23 日紧急电量从问题二的 892.40 kWh 降至 170.85 kWh，总费下降 3909.92 元；3 月 20 日日内方案却比问题二多支出 109.40 元，说明调整机会并不保证每一天都降低费用。图3展示 9 月 23 日的原计划、最终购电与实际净需求；竖虚线标出三次允许修订的时刻。最终购电在原计划上补充部分缺口，后续库存因此改变，紧急购电的规模随之下降。

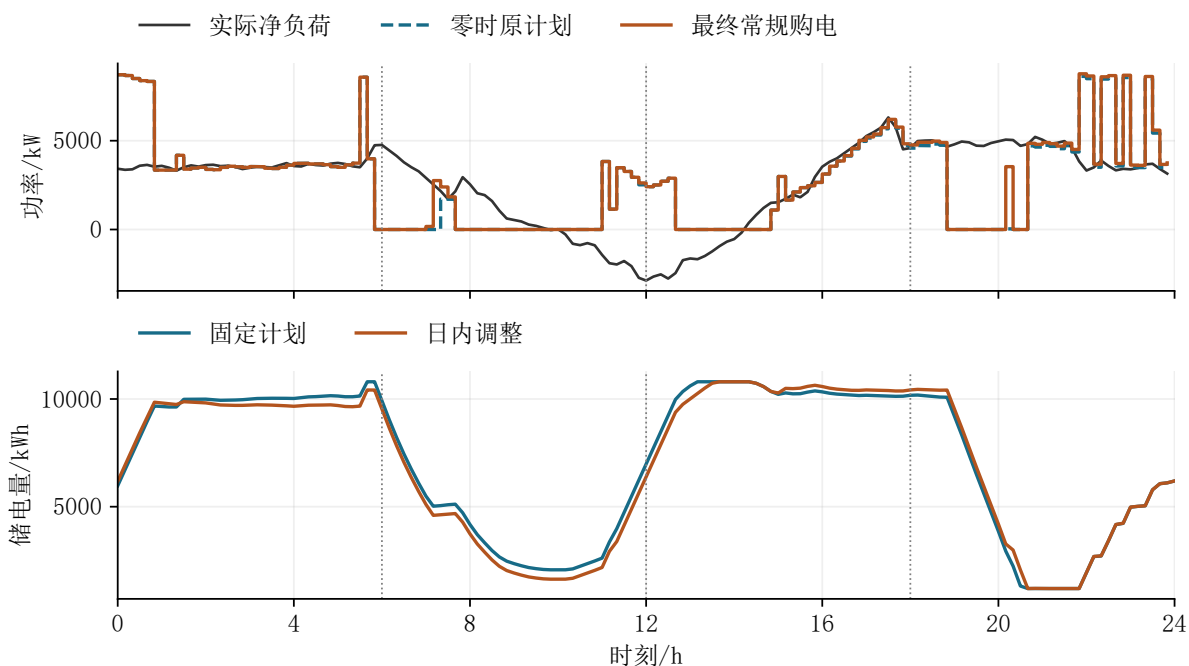


图3 9月23日的日内购电调整与库存变化

5.3.6 预报贡献与参数灵敏度

先比较不同光伏来源。各来源在相同1月区间选参，正式期共用热启动和期初库存。融合方案比纯历史基准节省112379.91元，纯附件三反而较贵，因而保留历史曲线。完整费用见附录B.1表58；这种组合比较尚不能单独识别某次新预报的价值。

为回答是否引入其他时刻预报，固定主参数、三次重规划、负荷修正及当前光伏锚点，仅切换允许读取的新预报；未读取时沿用最近可用发布行。各子集共用1月热启动，之后连续演化库存。表13列出单独撤去一次新预报的条件费用差，八个子集的完整结果见附录B.1表56。

表13 单独撤去一次新小时预报的条件费用差（元）

撤去的预报	纯附件3	融合主策略
06点	53,593.3113	28,402.5518
12点	74,628.9683	20,127.5241
18点	-0.1296	-0.3123

全部新预报相对全部不读取节省17225.82元。撤去06或12点新预报会增加费用，撤去18点的变化仅-0.31元。前两次预报在当前组合下有可辨认的价值，18点新预报未显示经济增益。这些边际量有交互，不能相加；对照中三次调整机会仍保留，也不能据此取消18点调整购电。

分位扰动见表14。正式期 $\alpha = 0.50$ 费用比主参数0.65低15787.50元，但紧急电量

明显增大。该结果暴露了 1 月短评分窗口对全年选择的局限，本文保留按 1 月选出的主参数，不在观察全年后追认更便宜的参数为预先选定结果。

表 14 问题三的分位参数扰动（2—12 月）

α	总费用/元	紧急购电/kWh	年末储电量/kWh
0.5	13,300,048.49	66,633.50	5,890.36
0.65	13,315,835.99	25,862.57	5,987.72
0.7	13,393,753.32	18,804.41	6,060.07
0.9	14,101,446.70	2,544.90	6,507.64

进一步将日末预测恢复目标改为 3000 或 9000 kWh，结果见附录B.1表57。目标较低时现金费用下降，但热启动库存和期末库存也降低；不能把费用差全视为运行效率改进。

逐笔成交不可撤回时，重新选参和优化的总费为 13,460,502.62 元，仍比问题二低 447673.33 元，节省少于主合约。附录B.2分别列出冻结路径重计与重新优化，说明收益对交易累计规则的依赖。

5.4 问题四：波动电价下的规划

5.4.1 价格信息与两种决策频率

问题四在相同负荷、光伏和储能条件下，将固定日内电价换为附件四的实时波动价格。不能只把问题二、三的购电量乘上新电价，因为价格会改变充放电和购电安排；也不能在每天零时提前读取当天全部实际电价，否则实际执行策略含有先知信息。动态价格研究也区分预测驱动的可实施控制与全知性能参考^[4]。

本文把“交易时刻电价”解释为对应交付区间的实际价格 $p_{d,t}$ 。该价格在实际交付和结算时可知，未来值则需要预测。沿用问题三的交易变量，记 $\hat{p}_{d,t}^{(s)}$ 为时刻 s 基于已到达信息得到的未来价格估计。问题 4-2 只允许 $s = 0$ 制定固定计划；问题 4-3 允许 $s \in \{0, 36, 72, 108\}$ 规划，并沿用光伏预报融合和最终净调整合约。

两种策略保持同一物理设备、十分钟步长、充放电反馈和连续库存约定。增加的只是价格预测，不引入题目未给出的输电容量、需求响应或储能寿命费用。

5.4.2 费用目标与价格预测

问题 4-2 和 4-3 的真实评价费用分别为

$$C_{4-2} = \sum_{d,t} p_{d,t}(g_{d,t} + 5e_{d,t}), \quad (30)$$

$$C_{4-3} = \sum_{d,t} p_{d,t} [g_{d,t} + 1.5(q_{d,t} - g_{d,t})_+ + 0.5(g_{d,t} - q_{d,t})_+ + 5e_{d,t}]. \quad (31)$$

求和均覆盖 2—12 月，计费系数为实际价格。在制定计划时，这些未来系数尚未知，因而零时和日内分别求解

$$\min \sum_{t=0}^{143} \hat{p}_{d,t}^{(0)} g_{d,t}, \quad \min \sum_{t=s}^{143} 1.5 \hat{p}_{d,t}^{(s)} (q_{d,t}^{(s)} - g_{d,t}). \quad (32)$$

第二个目标只用于问题 4-3 的 $s > 0$ 。规划目标用于选择动作，实际账单用于评价，不能将预测目标值作为最终费用报告。

价格预测沿用负荷的同星期加权结构。对 $d \geq 7$ ，定义历史价格曲线

$$\bar{p}_{d,t} = \frac{\sum_{k=1}^{K_d} 0.55^{k-1} p_{d-7k,t}}{\sum_{k=1}^{K_d} 0.55^{k-1}}, \quad K_d = \min\{4, \lfloor d/7 \rfloor\}. \quad (33)$$

不足一周时取此前最多 7 天，以 0.8 的衰减系数加权。零时取 $\hat{p}_{d,t}^{(0)} = \max(\bar{p}_{d,t}, 0.001)$ 。问题 4-3 在日内用最近 18 段已实现价格的均值修正水平，仍使用式(25)定义的截断函数，但将价格修正比例限制在 $[0.5, 1.5]$ 。记

$$r_{d,s}^p = \text{clip} \left(\frac{\frac{1}{18} \sum_{j=s-18}^{s-1} p_{d,j}}{\max \left(\frac{1}{18} \sum_{j=s-18}^{s-1} \bar{p}_{d,j}, 0.01 \right)}, 0.5, 1.5 \right), \quad (34)$$

则剩余预测为

$$\hat{p}_{d,t}^{(s)} = \max\{0.001, r_{d,s}^p \bar{p}_{d,t}\}, \quad t \geq s, \quad d \geq 1. \quad (35)$$

截断避免极小历史均值使比例失控，正下限保证购电系数非负。首日各发布时间统一沿用附件一给定的固定价格，不进行当日比例修正；该先验只参与 1 月初始化。预测不读取当前未完成区间或未来实际价格。

5.4.3 分条约束与汇总模型

a) 物理约束。每次规划仍满足风险净负荷下的能量平衡、储能效率、功率和安全库存限制。价格改变目标系数，不改变物理守恒关系。

b) 库存衔接。零时从实际 $S_{d,0}$ 开始，日内从实际 $S_{d,s}$ 开始，预测日末恢复目标均为 6000 kWh。实际跨日运行不重置库存。

c) 交易频率与原计划下界。问题 4-2 全天 $q_t = g_t$ ；问题 4-3 只在三次允许时刻改报未交付量，且 $q_t^{(s)} \geq g_t$ 。实际和预测价格均为正，问题三的不减购支配关系仍成立。

d) 可用信息与实际结算。预测价格、净负荷和本次计划均来自 $\mathcal{H}_{d,s}$ ；之后发生的 $p_{d,t}$ 只进入对应实际费用。即使事后发现价格预测偏差，也不回写已执行的决策。

为统一两种频率，令允许的发布时间集合为 $\mathcal{S}$ ；零时取 $\ell_t^{(0)} = 0$ 、 $\kappa_0 = 1$ ，日内取 $\ell_t^{(s)} = g_t$ 、 $\kappa_s = 1.5$ 。每次求解

$$\begin{aligned}
(\mathbf{P}_4^{(s)}) \quad & \min \sum_{t=s}^{143} \kappa_s \widehat{p}_{d,t}^{(s)} (q_t^{(s)} - \ell_t^{(s)}), \\
\text{s.t.} \quad & q_t^{(s)} + \widehat{b}_t = \widetilde{N}_{d,t}^{(s)} + \widehat{c}_t + \widehat{w}_t, \\
& \widehat{S}_{t+1} = \widehat{S}_t + 0.9\widehat{c}_t - \widehat{b}_t/0.9, \\
& 0 \leq \widehat{c}_t, \widehat{b}_t \leq 5000/6, \quad q_t^{(s)} \geq \ell_t^{(s)}, \quad \widehat{w}_t \geq 0, \\
& 1200 \leq \widehat{S}_t \leq 10800, \quad \widehat{S}_s = S_{d,s}, \quad \widehat{S}_{144} \geq 6000.
\end{aligned} \tag{36}$$

零时输出定义为 $g_t = q_t^{(0)}$ 。问题 4-2 取 $\mathcal{S} = \{0\}$ ，问题 4-3 取 $\mathcal{S} = \{0, 36, 72, 108\}$ ；区间约束作用于 $t = s, \dots, 143$ ，库存边界包括 144。这样明确列出两种模型的差异，无需为波动价格重新定义一套物理变量。

5.4.4 求解与价格到账顺序

沿用问题二、三的候选集合和 1 月 15—31 日现金评分规则。问题 4-2 选中 $\alpha = 0.80$ ，问题 4-3 选中 $(\lambda, \alpha) = (0.50, 0.65)$ 。问题 4-3 的 1 月热启动仍固定使用纯附件三、0.80 分位与三次调整，2 月才切换参数。

按时间先后运行以下步骤：

1. 零时继承真实库存，以过去日期分别预测负荷、光伏和价格；构造风险净负荷。
2. 用预测价格作为目标系数求解 $(\mathbf{P}_4^{(0)})$ ，锁定原计划并保存预测曲线。
3. 问题 4-2 不再修改常规计划；问题 4-3 在允许发布时刻更新已到达预报、负荷及价格水平，求解剩余时域模型。
4. 实际交付时以当段真实净负荷执行充放电与紧急补购，不将新观测传回此前决策。
5. 用该段实际价格结算原计划、最终净调整和紧急购电，更新实际库存。
6. 日末传递库存，2 月起累计正式费用；从逐段数组分别重算全年和指定日期账单。

改变未来价格数据不应改变当前的预测和计划，这是检验该步骤顺序的直接办法。离线预先生成预测数组只是计算缓存，每次使用仍按日期与发布时间限定，不能把缓存中保存了未来预测误差理解为当前可读取这些误差。

5.4.5 波动电价结果及比较

问题 4-2 总费用为 14,675,844.87 元，问题 4-3 为 14,044,885.25 元，日内方案节省 630,959.63 元，降幅 4.30%。表 15 给出费用与电量分解。

表 15 波动电价下两种策略的结果（2—12 月）

指标	问题 4-2	问题 4-3
原计划费/元	14,032,328.84	13,366,608.69
调整费/元	0.00	550,605.94
紧急费/元	643,516.03	127,670.62
总费用/元	14,675,844.87	14,044,885.25
常规购电/kWh	21,708,087.29	21,192,613.91
紧急购电/kWh	101,425.76	24,227.41
未用电量/kWh	2,632,806.82	2,009,834.69
发生紧急购电的天数	108	134
期初储电量/kWh	6,864.27	7,114.54
期末储电量/kWh	6,263.02	5,997.11

日内方案的原计划费和紧急费下降，新增 550605.94 元调整费后，总支出仍较低。紧急电量由 101425.76 kWh 降至 24227.41 kWh，但发生紧急电的天数由 108 增至 134，与固定价格下的现象一致：总量改善与出现频率不是同一个指标。

图4给出逐月费用及紧急电量。费用比较应在同一月份、同一价格制度下进行；直接将固定价格总费与波动价格总费相减，同时包含价格水平、购电路径和库存变化，不能作为价格预测误差的估计。

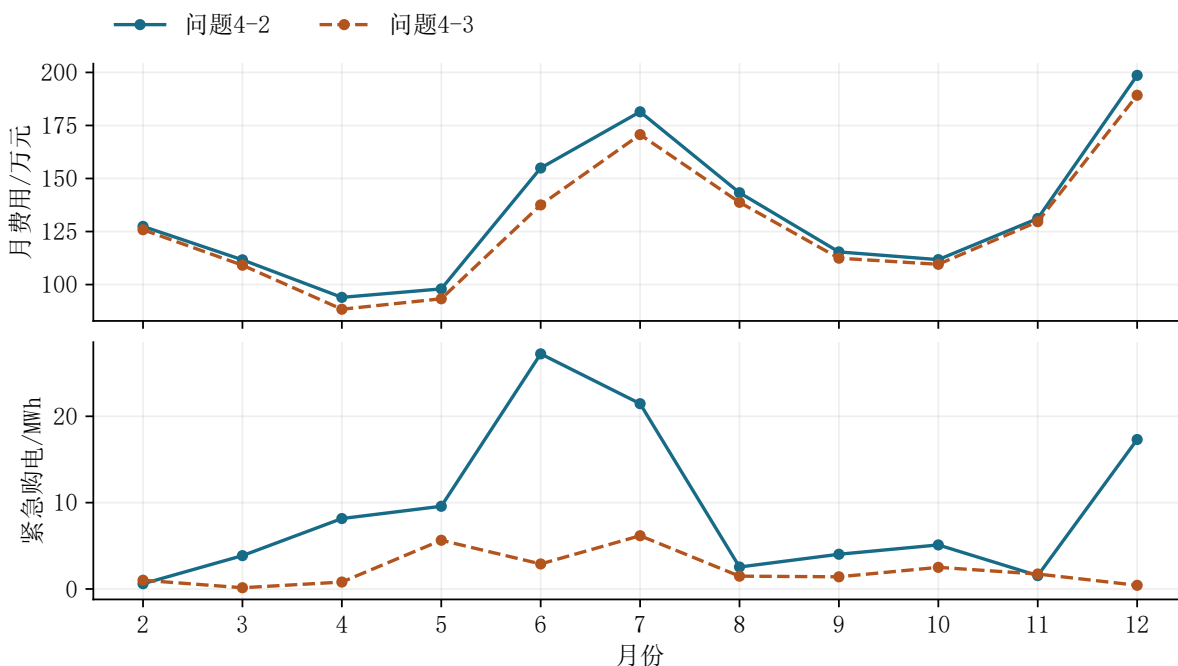


图 4 波动电价下各月费用与紧急购电量

指定日期费用列于表16。在 9 月 23 日，允许调整后的全天总费从 49762.23 元降至 45799.34 元；12 月 21 日两种方案费用均较高。下一小节分别展示 9 月 23 日的购电与储能；其余日期、紧急事件及账单见附录A.3、A.4，完整策略保存在 result4-2.xlsx、result4-3.xlsx。

表 16 波动电价下指定日期费用（2025 年，元）

日期	问题 4-2	问题 4-3	日内方案节省
03-20	42,627.22	42,600.22	27.00
06-21	19,640.47	18,336.05	1,304.42
09-23	49,762.23	45,799.34	3,962.89
12-21	75,769.21	72,885.14	2,884.07

采用共同热启动比较光伏来源，融合方案比纯历史基准少支出 122075.92 元，纯附件三仍未优于融合，完整结果见附录B.1表59。各预测器分别选参，该差额属于预测与控制组合的经济差异。

5.4.6 代表日期的购电与储能结果

a) 问题 4-2：零时固定计划。以下按题面表 1、表 2 列出 9 月 23 日。购电表给出常规计划电量及按实际交付价格结算的常规费；电量为 kWh，费用为元。2025.9.23 的购电和储能结果分别见表17和表18。

表 17 微网在指定时间段的购电量及全天的购电量和购电费（2025.9.23）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	499.87	14:00–14:10	0.00
16:00–16:10	531.67	18:00–18:10	775.50	20:00–20:10	0.00
全天购电量	68,171.89	全天购电费	44,011.52		

表 18 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.9.23）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,837.74	54.89	4:00–8:00	801.19	6,062.63
8:00–12:00	5,459.91	1,896.62	12:00–16:00	4,418.80	560.16
16:00–20:00	79.59	5,673.19	20:00–24:00	5,585.42	2,617.77
0:00 储电量	5,901.66	24:00 储电量	6,226.87		

b) 问题 4-3：日内调整。购电表分别展示零时原计划与最终常规购电，费用口径与问题三相同，均按实际交付价格核算；储能表为实际执行值。两组结果承接各自连续运行

的真实库存，不将指定日重置为 6000 kWh。2025.9.23 的购电和储能结果分别见表19和表20。

表 19 微网在指定时间段的购电量及全天的购电量和购电费（2025.9.23）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	401.83	14:00–14:10	0.00
16:00–16:10	520.87	18:00–18:10	763.88	20:00–20:10	0.00
全天购电量		65,294.82	全天购电费		42,202.79
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	401.83	14:00–14:10	0.00
16:00–16:10	520.87	18:00–18:10	793.71	20:00–20:10	0.00
全天购电量		67,099.79	全天购电费		44,903.59

表 20 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.9.23）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,267.49	194.62	4:00–8:00	996.54	6,251.03
8:00–12:00	5,281.76	1,896.62	12:00–16:00	5,288.36	509.61
16:00–20:00	111.95	5,255.66	20:00–24:00	5,591.15	3,287.90
0:00 储电量		6,169.40	24:00 储电量		6,224.67

5.4.7 价格与参数灵敏度检验

首先针对价格预测形状作局部扰动。记 $m_{d,s}$ 为本次剩余时域预测价格均值，将价格曲线改为

$$\hat{p}_{d,t}^{(s,\theta)} = \max\{0.001, m_{d,s} + (1 + \theta)(\hat{p}_{d,t}^{(s)} - m_{d,s})\}, \quad \theta \in \{-0.2, 0, 0.2\}. \quad (37)$$

负扰动缩小预测峰谷差，正扰动扩大预测峰谷差。该变换只使用当时可得的预测曲线，不加入未来真值。负荷、光伏、实际结算价、控制参数和共同 1 月热启动保持一致，从 2 月起重新规划并连续执行。表21列出实际费用和年末库存。

表 21 预测价差幅度扰动（共同热启动，2—12 月）

方案	θ	总费用/元	较主方案增费/元	期末储电量/kWh
问题 4-2	-0.2	14,674,479.32	-1,365.55	6,263.02
问题 4-2	0	14,675,844.87	0.00	6,263.02
问题 4-2	+0.2	14,683,290.13	7,445.26	6,263.02
问题 4-3	-0.2	14,042,823.76	-2,061.48	5,997.11
问题 4-3	0	14,044,885.25	0.00	5,997.11
问题 4-3	+0.2	14,061,149.66	16,264.42	5,997.11

扩大预测价差 20% 时，两种策略分别增费 7445.26 元和 16264.42 元；缩小 20% 时分别减费 1365.55 元和 2061.48 元。四个扰动路径与各自主方案的期末库存一致，因此这次费用差不由边界库存不同造成。在本次范围内，总费变化均小于 0.12%，日内调整方案仍较低；但这只检验价差幅度误差，没有覆盖峰谷时刻错位或极端电价冲击。

若直接把未来实际价格作为规划输入，问题 4-2 与 4-3 费用分别为 14,566,416.06 元和 13,932,754.85 元。这个诊断改变了可用信息，并伴随热启动和库存路径变化；它不是可执行策略，也不是随机全年最优值的严格下界。具体预测策略中约 0.04456 元/kWh 的零时平均绝对价格误差，同样不能直接换算为这项费用差。

其次检查风险参数选择。附录 B.1 表 60 报告 1 月 15—31 日评分，问题 4-3 固定融合权重 0.50。固定计划在 0.80 处、日内调整在 0.65 处取得本组最低评分；较高分位减少缺口，却增加提前购买的支出。该结果属于选参窗口，不是未见年度的敏感性证据。

跨月实验按训练先于评价的顺序^[1]，仅以上月费用重选参数，并从共同 4 月 1 日库存连续运行。问题 4-3 比固定参数多支出 16320.66 元，9 个月中仅 1 个月更省；完整费用与库存见附录 B.4。同年滚动选参未自动改善累计费用。

另以确定性滚动反馈^[3]作同信息、同参数及共同热启动对照，问题 4-2 和 4-3 分别增费 33995.79 元、213489.58 元，期末库存与原反馈一致。该配置未校准跨日库存价值，不能据此证明原反馈最优；模型及费用分解见附录 B.3。

六、 结果分析与模型检验

6.1 主策略费用汇总

四种主策略均按 2025 年 2—12 月 334 天统计，完整费用分解见表 22。原计划费、调整费和紧急费只各计一次；问题三相对问题二、问题 4-3 相对问题 4-2 的降费率分别为 4.26% 和 4.30%。固定价格与波动价格之间的总费用差同时含有价格水平和调度路径的变化，不用于评价价格预测误差。

表 22 四种主方案的正式期费用（2025 年 2—12 月，元）

方案	原计划费用	调整费用	紧急费用	总费用
问题 2	13,291,625.72	0.00	616,550.23	13,908,175.95
问题 3	12,649,467.17	527,937.93	138,430.89	13,315,835.99
问题 4-2	14,032,328.84	0.00	643,516.03	14,675,844.87
问题 4-3	13,366,608.69	550,605.94	127,670.62	14,044,885.25

6.2 物理、信息与账单核验

各问已分别报告目标费用、指定日期和灵敏度结果。本节统一检查输出是否实现了所列约束。对每个交付区间，独立从归档数组计算

$$\begin{aligned} r_{d,t}^E &= q_{d,t} + e_{d,t} + V_{d,t} + b_{d,t} - L_{d,t} - c_{d,t} - w_{d,t}, \\ r_{d,t}^S &= S_{d,t+1} - S_{d,t} - 0.9c_{d,t} + b_{d,t}/0.9. \end{aligned} \quad (38)$$

两类残差在设定的 10^{-6} kWh 容差内，并逐段检查库存、功率、互斥及跨日连续性。账单由原计划、最终调整和紧急购电分别重算，检查与汇总及指定日期工作簿一致；不求解器返回成功代替这些检查。

信息检验通过改变尚未发生的实测或尚未发布的预报，确认当前预测与计划不变，并以改变已到达信息作正向对照。既有主调度、预报合约、反馈跨月验证分别通过 179、1134、2962 项检查；本轮新增容量、功率、效率和价差扰动另通过 201 项独立检查。这些检查支持可行性、信息顺序和计费一致，只有问题一另具 LP 与 MILP 最优性证据。

6.3 补充对照及结论边界

逐笔收费、价格感知反馈及上月选参的完整账单分别见附录 B.2、B.3、B.4。逐笔制度下重新优化后，固定与波动价格的日内方案分别节省 447673.33 元、475880.51 元，少于主合约；四组滚动反馈均因紧急费增加而增费。逐月融合在问题三、4-3 分别多支出 2109.14 元、16320.66 元。上述对照保留各自的信息、选参窗口与边界库存，说明复杂反馈和频繁选参未必更省；同年结果不能替代未见年度验证。

七、 模型的评价与推广

7.1 模型的优点和不足

模型的主要优点可归纳为以下三点。

1. 运行与账单逐段对应。能量平衡、损耗和库存约束统一建模，真实反馈与交易结算独立核验；指定日期和全年费用均可追溯至十分钟记录。

2. **信息条件清楚。**预测只用已完成观测和已发布预报，跨日库存连续承接；信息子集对照区分了新预报与重规划机会的作用。
3. **求解与解释较简洁。**储能规划采用线性模型，日内规模随剩余时域缩短；设备、分位和合约检验分别解释费用变化，问题一另有整数模型复核。

模型的不足主要体现在以下三个方面。

1. **风险与库存价值处理较简化。**逐段残差分位未联合刻画时间相关性，也不等于可靠率；日末恢复目标与当前缺口优先反馈尚未联合优化跨日库存价值。
2. **参数具有样本依赖。**1月短窗口选出的分位未必全年最省，逐月重选与所测试的滚动反馈也未改善累计费用。现有经济结果属于2025年再分析，不能直接外推。
3. **实际运行条件尚未完整纳入。**模型未计电池折损、外网容量及响应延迟，并假定紧急电可补足缺口；追加购电是否允许撤回，也会改变费用与策略。

7.2 模型的推广

后续推广可沿数据、设备和控制三个方向逐步开展。

1. **其他年度与负荷类型。**替换负荷、光伏和电价输入，按实际发布时间对齐；在未参与开发的数据上，冻结模型族与选参规则后连续回测，同时比较费用、紧急电量和边界库存。
2. **供电限制与寿命管理。**有设备资料后，加入购电功率上限、循环吞吐成本和随健康状况变化的容量。紧急电也受限时需记录缺供量并设置代价；扩容评价还应计入投资成本。
3. **多时段风险控制。**构造保留误差时间相关性的场景或不确定集合，在跨日滚动中估计末端库存价值。相关研究^[5,12]可提供方法背景，仍须在相同信息、合约和初始库存下比较经济效果与计算开销。这些方向属于待验证扩展。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于数据处理与建模分析、代码生成与调试、文献检索与整理、计算结果核对、论文撰写与排版，详细使用情况见支撑材料。

参考文献

- [1] 吴杰,温晨阳,李珊,等. 基于分时电价的光伏-储能系统容量优化配置[J/OL]. 电工电能新技术, 2018, 37 (1): 23-30. <https://ateee.iee.ac.cn/CN/abstract/abstract1702.shtml>. DOI:10.12067/ATEEE1704027.

- [2] 肖海伟, 孙久严, 李章溢, 等. 基于综合储能设备的工业园区联络线功率实时 MPC 控制方法[J/OL]. 电力建设, 2019, 40 (8): 34-42. <https://www.cepc.com.cn/CN/10.3969/j.issn.1000-7229.2019.08.005>.
- [3] Wytock M, Moehle N, Boyd S. Dynamic energy management with scenario-based robust MPC[C/OL]//2017 american control conference (ACC) . Seattle, WA, USA: IEEE, 2017: 2042-2047. https://web.stanford.edu/~boyd/papers/pdf/dyn_ener_man_acc.pdf.
- [4] Perez-Pineiro D, Skogestad S, Boyd S. Home energy management with dynamic tariffs and tiered peak power charges [Manuscript][EB/OL]. (2023) [2026-09-10]. <https://stanford.edu/~boyd/papers/hem.html>.
- [5] 李嘉伟, 巨云涛, 张璐, 等. 基于分布鲁棒模型预测控制的微电网多时间尺度优化调度[J/OL]. 电力工程技术, 2024, 43 (4): 45-55. <https://epet-info.com/html/2024/4/230524600.html>. DOI:10.12158/j.2096-3203.2024.04.005.
- [6] The SciPy community. linprog (method='highs'): SciPy reference guide[EB/OL]. [2026-09-10]. <https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html>.
- [7] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J/OL]. Mathematical Programming Computation, 2018, 10 (1): 119-142. <https://link.springer.com/article/10.1007/s12532-017-0130-5>.
- [8] 胡俊杰, 曲佳桐, 刘雪涛, 等. 考虑应急电源车的高速公路服务区微网两阶段优化调度方法[J/OL]. 电力建设, 2025, 46 (4): 150-159. <https://www.cepc.com.cn/CN/10.12204/j.issn.1000-7229.2025.04.013>.
- [9] Bates J M, Granger C W J. The combination of forecasts[J/OL]. Journal of the Operational Research Society, 1969, 20 (4): 451-468. <https://www.tandfonline.com/doi/abs/10.1057/jors.1969.103>.
- [10] 隗知初, 杨苹, 周钱雨凡, 等. 基于 Q-Learning 的多模态自适应光伏功率优化组合预测[J/OL]. 电力工程技术, 2026, 45 (1): 115-124,163. <https://epet-info.com/html/2026/1/20260110.html>. DOI:10.12158/j.2096-3203.2026.01.011.
- [11] Cerqueira V, Torgo L, Mozetič I. Evaluating time series forecasting models: an empirical study on performance estimation methods[J/OL]. Machine Learning, 2020, 109 (11): 1997-2028. <https://link.springer.com/article/10.1007/s10994-020-05910-7>.
- [12] 李佳玮, 孙庆贺, 王琼, 等. 计及不确定相关性的多能微电网数据驱动分布鲁棒优化调度[J/OL]. 电力建设, 2025, 46 (8): 22-33. <https://www.cepc.com.cn/CN/10.12204/j.issn.1000-7229.2025.08.003>.

A 指定日期的补充结果

正文保留四日期汇总及 9 月 23 日购电、储能原格式表。本附录列出其余三个日期，并按题面表 3 并列四日期全部紧急事件，另列完整账单；电量为 kWh、费用为元，计量与舍入口径见第 5.2 节。

A.1 问题二

9 月 23 日购电与储能见正文表 6、7。2025.3.20 的购电和储能结果分别见表 23 和表 24。

表 23 微网在指定时间段的购电量及全天的购电量和购电费（2025.3.20）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	605.05	14:00–14:10	0.00
16:00–16:10	493.66	18:00–18:10	711.53	20:00–20:10	0.00
全天购电量		68,386.72		全天购电费	
				41,537.79	

表 24 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.3.20）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	3,932.12	143.66	4:00–8:00	1,068.89	5,197.90
8:00–12:00	6,214.23	2,777.38	12:00–16:00	3,718.36	415.81
16:00–20:00	261.15	5,734.03	20:00–24:00	5,662.99	2,950.27
0:00 储电量		6,663.82	24:00 储电量		6,303.52

2025.6.21 的购电和储能结果分别见表 25 和表 26。

表 25 微网在指定时间段的购电量及全天的购电量和购电费（2025.6.21）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	247.19	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量		37,707.19		全天购电费	
				22,414.49	

表 26 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.6.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	1,016.02	1,120.01	4:00–8:00	437.87	4,390.39
8:00–12:00	8,456.58	0.00	12:00–16:00	0.00	0.00
16:00–20:00	312.75	5,157.72	20:00–24:00	6,563.91	2,485.90
0:00 储电量		8,003.24	24:00 储电量		8,496.09

2025.12.21 的购电和储能结果分别见表27和表28。

表 27 微网在指定时间段的购电量及全天的购电量和购电费（2025.12.21）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	994.23	14:00–14:10	0.00
16:00–16:10	855.14	18:00–18:10	733.64	20:00–20:10	0.00
全天购电量		99,788.74	全天购电费		64,446.61

表 28 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.12.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,514.59	57.06	4:00–8:00	910.22	1,414.62
8:00–12:00	5,914.67	6,794.16	12:00–16:00	6,186.24	2,220.11
16:00–20:00	0.00	5,577.31	20:00–24:00	5,753.40	2,842.53
0:00 储电量		6,677.89	24:00 储电量		6,622.69

表29按题面表 3 横向列出四个日期的全部紧急购电事件，表30给出含紧急费的完整账单。

表 29 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.00	无	0.00	20:20–21:50	892.40	无	0.00
全天合计	0.00	全天合计	0.00	全天合计	892.40	全天合计	0.00

表 30 问题 2 指定日期的全天账单（元）

日期	原计划费	调整费	紧急费	总费用
03-20	41,537.79	0.00	0.00	41,537.79
06-21	22,414.49	0.00	0.00	22,414.49
09-23	42,079.63	0.00	5,734.95	47,814.59
12-21	64,446.61	0.00	0.00	64,446.61

A.2 问题三

9 月 23 日购电与储能见正文表11、12。购电表分别列零时原计划与最终常规购电，后者费用含调整费。2025.3.20 的购电和储能结果分别见表31和表32。

表 31 微网在指定时间段的购电量及全天的购电量和购电费（2025.3.20）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	557.73	14:00–14:10	0.00
16:00–16:10	553.12	18:00–18:10	708.74	20:00–20:10	0.00
全天购电量		66,614.07	全天购电费		40,394.87
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	557.73	14:00–14:10	0.00
16:00–16:10	553.12	18:00–18:10	708.74	20:00–20:10	0.00
全天购电量		67,646.32	全天购电费		41,647.19

表 32 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.3.20）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	3,934.61	284.62	4:00–8:00	1,085.82	5,512.63
8:00–12:00	5,764.68	2,777.38	12:00–16:00	4,814.41	262.75
16:00–20:00	26.19	5,714.39	20:00–24:00	5,567.69	2,988.71
0:00 储电量		6,570.81	24:00 储电量		6,155.47

2025.6.21 的购电和储能结果分别见表33和表34。

表 33 微网在指定时间段的购电量及全天的购电量和购电费（2025.6.21）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	204.64	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量		35,822.67	全天购电费		20,860.00
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	204.64	18:00–18:10	0.00	20:00–20:10	0.00
全天购电量		35,854.93	全天购电费		20,909.49

表 34 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.6.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	614.20	920.51	4:00–8:00	421.40	4,390.39
8:00–12:00	9,383.47	0.00	12:00–16:00	0.00	0.00
16:00–20:00	158.82	5,371.22	20:00–24:00	6,203.59	2,485.90
0:00 储电量		7,323.84	24:00 储电量		7,796.04

2025.12.21 的购电和储能结果分别见表35和表36。

表 35 微网在指定时间段的购电量及全天的购电量和购电费（2025.12.21）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	959.50	14:00–14:10	0.00
16:00–16:10	849.15	18:00–18:10	723.11	20:00–20:10	0.00
全天购电量		96,792.11	全天购电费		62,109.93
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	959.50	14:00–14:10	0.00
16:00–16:10	849.15	18:00–18:10	723.11	20:00–20:10	0.00
全天购电量		96,803.32	全天购电费		62,129.52

表 36 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.12.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,565.52	131.98	4:00–8:00	1,321.73	1,481.00
8:00–12:00	5,703.78	7,129.08	12:00–16:00	6,932.53	2,283.94
16:00–20:00	10.53	5,708.00	20:00–24:00	5,587.69	2,843.50
0:00 储电量		6,379.90	24:00 储电量		6,336.73

表37按题面表 3 横向列出四个日期的全部紧急购电事件，表38给出含紧急费的完整账单。

表 37 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.00	无	0.00	20:20–20:40	15.16	无	0.00
				20:50–21:10	90.07		
				21:20–21:50	65.62		
全天合计	0.00	全天合计	0.00	全天合计	170.85	全天合计	0.00

表 38 问题 3 指定日期的全天账单（元）

日期	原计划费	调整费	紧急费	总费用
03-20	40,394.87	1,252.32	0.00	41,647.19
06-21	20,860.00	49.49	0.00	20,909.49
09-23	40,305.24	2,779.69	819.75	43,904.67
12-21	62,109.93	19.59	0.00	62,129.52

A.3 问题 4-2

9 月 23 日购电与储能见正文表17、18。各项费用按实际交付价格结算。2025.3.20 的购电和储能结果分别见表39和表40。

表 39 微网在指定时间段的购电量及全天的购电量和购电费（2025.3.20）

时间段	购电量	时间段	购电量	时间段	购电量	
10:00–10:10	0.00	12:00–12:10	605.05	14:00–14:10	0.00	
16:00–16:10	493.66	18:00–18:10	711.53	20:00–20:10	271.08	
全天购电量		68,314.47		全天购电费		42,627.22

表 40 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.3.20）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,477.53	96.73	4:00–8:00	776.94	5,581.54
8:00–12:00	6,214.23	2,777.38	12:00–16:00	3,718.36	415.81
16:00–20:00	206.46	5,950.42	20:00–24:00	5,735.02	2,666.85
0:00 储电量		6,809.84	24:00 储电量		6,393.60

2025.6.21 的购电和储能结果分别见表41和表42。

表 41 微网在指定时间段的购电量及全天的购电量和购电费（2025.6.21）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	247.19	18:00–18:10	518.51	20:00–20:10	0.00
全天购电量		38,164.95	全天购电费		19,640.47

表 42 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.6.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	766.73	4,213.83	4:00–8:00	1,369.00	1,719.95
8:00–12:00	8,448.53	0.00	12:00–16:00	0.00	0.00
16:00–20:00	0.00	5,185.01	20:00–24:00	7,302.26	3,091.92
0:00 储电量		7,867.26	24:00 储电量		8,175.44

2025.12.21 的购电和储能结果分别见表43和表44。

表 43 微网在指定时间段的购电量及全天的购电量和购电费（2025.12.21）

时间段	购电量	时间段	购电量	时间段	购电量	
10:00–10:10	0.00	12:00–12:10	994.23	14:00–14:10	0.00	
16:00–16:10	855.14	18:00–18:10	733.64	20:00–20:10	0.00	
全天购电量		99,790.43		全天购电费		75,769.21

表 44 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.12.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,580.59	57.06	4:00–8:00	809.79	1,385.35
8:00–12:00	5,931.88	6,811.36	12:00–16:00	6,190.28	2,220.11
16:00–20:00	0.00	5,577.31	20:00–24:00	5,748.19	2,842.53
0:00 储电量		6,676.37	24:00 储电量		6,618.00

表45按题面表 3 横向列出四个日期的全部紧急购电事件，表46给出含紧急费的完整账单。

表 45 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.00	无	0.00	18:20–18:30	1.08	无	0.00
				20:10–22:00	900.32		
全天合计	0.00	全天合计	0.00	全天合计	901.41	全天合计	0.00

表 46 问题 4-2 指定日期的全天账单（元）

日期	原计划费	调整费	紧急费	总费用
03-20	42,627.22	0.00	0.00	42,627.22
06-21	19,640.47	0.00	0.00	19,640.47
09-23	44,011.52	0.00	5,750.71	49,762.23
12-21	75,769.21	0.00	0.00	75,769.21

A.4 问题 4-3

9 月 23 日购电与储能见正文表19、20。原计划与最终购电分列，费用均按实际交付价格结算。2025.3.20 的购电和储能结果分别见表47和表48。

表 47 微网在指定时间段的购电量及全天的购电量和购电费（2025.3.20）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	557.73	14:00–14:10	0.00
16:00–16:10	553.12	18:00–18:10	708.74	20:00–20:10	169.45
全天购电量		66,482.67	全天购电费		41,467.81
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	557.73	14:00–14:10	0.00
16:00–16:10	553.12	18:00–18:10	708.74	20:00–20:10	169.45
全天购电量		67,423.97	全天购电费		42,600.22

表 48 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.3.20）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,611.97	198.08	4:00–8:00	297.63	5,615.82
8:00–12:00	5,764.68	2,777.38	12:00–16:00	4,814.41	262.75
16:00–20:00	21.19	5,841.32	20:00–24:00	5,613.95	2,794.52
0:00 储电量		6,689.07	24:00 储电量		6,267.33

2025.6.21 的购电和储能结果分别见表49和表50。

表 49 微网在指定时间段的购电量及全天的购电量和购电费（2025.6.21）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	204.64	18:00–18:10	489.64	20:00–20:10	0.00
全天购电量		36,155.25	全天购电费		18,303.04
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	204.64	18:00–18:10	489.64	20:00–20:10	0.00
全天购电量		36,184.92	全天购电费		18,336.05

表 50 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.6.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	510.14	4,223.53	4:00–8:00	1,295.43	1,721.18
8:00–12:00	9,399.81	0.00	12:00–16:00	0.00	0.00
16:00–20:00	0.00	5,385.41	20:00–24:00	6,970.19	3,091.92
0:00 储电量		7,320.39	24:00 储电量		7,653.92

2025.12.21 的购电和储能结果分别见表51和表52。

表 51 微网在指定时间段的购电量及全天的购电量和购电费（2025.12.21）

零时原计划					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	959.50	14:00–14:10	0.00
16:00–16:10	849.15	18:00–18:10	723.11	20:00–20:10	0.00
全天购电量		96,784.31	全天购电费		72,861.20
最终常规购电					
时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	959.50	14:00–14:10	0.00
16:00–16:10	849.15	18:00–18:10	723.11	20:00–20:10	0.00
全天购电量		96,795.52	全天购电费		72,885.14

表 52 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（2025.12.21）

时间段	充电量	放电量	时间段	充电量	放电量
0:00–4:00	4,626.56	131.98	4:00–8:00	1,228.44	1,461.20
8:00–12:00	5,780.01	7,205.31	12:00–16:00	6,950.41	2,283.94
16:00–20:00	10.53	5,708.00	20:00–24:00	5,611.05	2,857.70
0:00 储电量		6,386.93	24:00 储电量		6,341.96

表53按题面表 3 横向列出四个日期的全部紧急购电事件，表54给出含紧急费的完整账单。

表 53 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.00	无	0.00	18:20–18:30	1.08	无	0.00
				20:10–20:30	16.61		
				20:50–21:10	87.54		
				21:20–22:00	76.78		
全天合计	0.00	全天合计	0.00	全天合计	182.01	全天合计	0.00

表 54 问题 4-3 指定日期的全天账单（元）

日期	原计划费	调整费	紧急费	总费用
03-20	41,467.81	1,132.41	0.00	42,600.22
06-21	18,303.04	33.01	0.00	18,336.05
09-23	42,202.79	2,700.80	895.75	45,799.34
12-21	72,861.20	23.94	0.00	72,885.14

B 补充对照与参数明细

B.1 设备与预测参数检验

表55列出问题一的设备扰动结果，各组保持相同输入曲线与 6000 kWh 首末库存，每次只改变一个参数。

表 55 问题一的单参数扰动与最低电费

扰动参数	取值	费用/元	费用变化/%
可用容量/kWh	7680	36,704.73	4.49
可用容量/kWh	9600	35,126.95	0.00
可用容量/kWh	11520	33,868.53	-3.58
功率上限/kW	4000	35,247.87	0.34
功率上限/kW	5000	35,126.95	0.00
功率上限/kW	6000	35,083.29	-0.12
单向效率	0.85	36,603.41	4.20
单向效率	0.9	35,126.95	0.00
单向效率	0.95	33,767.03	-3.87

表56保留全部八种预报子集；各组保持主参数、三次重规划及共同热启动，仅改变允许读取的新预报。

表 56 问题 3 新小时光伏预报的完整子集对照

新增预报时刻	纯附件 3 费用（元）	融合策略费用（元）	融合紧急电（kWh）
无	13,580,285.75	13,333,061.82	29,178.88
18	13,580,285.30	13,333,062.82	29,178.86
12	13,564,801.30	13,344,238.23	29,324.32
12、18	13,564,801.43	13,344,238.54	29,324.42
06	13,585,837.75	13,335,964.19	25,744.14
06、18	13,585,837.09	13,335,963.52	25,744.09
06、12	13,511,207.99	13,315,835.68	25,862.47
06、12、18	13,511,208.12	13,315,835.99	25,862.57

表57比较日末预测恢复目标，费用差伴随边界库存变化。

表 57 日末预测储备目标的变化（2—12 月）

目标/kWh	总费用/元	期初储电量/kWh	期末储电量/kWh
3000	13,298,180.06	4,231.74	3,105.48
6000	13,315,835.99	7,089.87	5,987.72
9000	13,346,622.47	9,942.13	8,916.25

表58、59分别比较固定与波动电价下的光伏来源，各组使用共同热启动；表60为 1 月 15—31 日选参费用，问题 4-3 固定融合权重 0.50。

表 58 问题三的光伏预测基准（共同热启动，2—12 月）

预测来源	λ	α	总费用/元
纯历史	0	0.65	13,428,215.91
纯附件三	1	0.65	13,511,208.12
融合	0.5	0.65	13,315,835.99

表 59 问题 4-3 的光伏预测基准（共同热启动，2—12 月）

预测来源	λ	α	总费用/元
纯历史	0	0.65	14,166,961.16
纯附件三	1	0.65	14,258,363.92
融合	0.5	0.65	14,044,885.25

表 60 波动电价的 1 月分位评分（1 月 15—31 日）

α	问题 4-2 费用/元	问题 4-3 费用/元
0.5	1,029,631.23	964,022.95
0.65	985,076.81	957,308.74
0.7	974,056.08	959,947.92
0.8	966,879.67	972,861.43
0.9	991,846.47	996,125.45
0.95	1,013,304.20	1,015,980.29

B.2 逐笔收费的替代合约

若日内增购成交后不可免费撤回，需逐次记录修订。令 $q^{(0)} = g$ ，第 r 次修订为 $q^{(r)}$ ，则在适用交付段收取

$$C^{\text{trade}} = \sum_{d,t} \left[p_{d,t} g_{d,t} + 5 p_{d,t} e_{d,t} + \sum_r \left(1.5 p_{d,t} \left(q_{d,t}^{(r)} - q_{d,t}^{(r-1)} \right)_+ + 0.5 p_{d,t} \left(q_{d,t}^{(r-1)} - q_{d,t}^{(r)} \right)_+ \right) \right]. \tag{39}$$

这里 $p_{d,t}$ 仍按交付段结算。因原购电款不返还且余电可以不利用，减购先前承诺没有优势，替代规划采用 $q_t^{(r)} \geq q_t^{(r-1)}$ ，只将新增量逐笔计费。

表61区分主合约、冻结主调度的逐笔重计、逐笔制度下重新优化三种情形。冻结重计只更换账单规则，保留原调度与库存；重优化则从 1 月重新选参和运行，会改变热启动与后续路径，参数和首末库存见表62。

表 61 合约解释与相应策略费用（2—12 月，元）

方案	结算与策略	总费用	较无调整方案节省
问题 3	主合约	13,315,835.99	592,339.96
问题 3	冻结主调度，逐笔重计	13,642,823.29	265,352.66
问题 3	逐笔合约重新优化	13,460,502.62	447,673.33
问题 4-3	主合约	14,044,885.25	630,959.63
问题 4-3	冻结主调度，逐笔重计	14,390,498.18	285,346.69
问题 4-3	逐笔合约重新优化	14,199,964.36	475,880.51

表 62 逐笔合约重优化的参数与库存（电量单位：kWh）

方案	λ	α	紧急购电	2 月初储电量	年末储电量
问题 3	0.25	0.65	27,362.36	7,089.87	5,987.72
问题 4-3	0.25	0.65	27,071.28	7,114.54	5,997.11

重新优化后，固定与波动价格下分别比对应无调整方案节省 447673.33 元、475880.51 元，均低于主合约的节省。冻结重计费用更高，说明合约解释会影响经济结论；但其差额不能替代新制度下重新优化的结果。

B.3 价格感知储能反馈对照

为检验优先补足当前缺口的反馈是否足够，在每个十分钟段固定剩余常规承诺 q ，使用最新已允许时刻的预测，将首段净负荷和价格换为当段已知真值，求解

$$\min \sum_{k=t}^{143} 5\hat{p}_k e_k, \quad c_k \leq \left(q_k - \tilde{N}_k\right)_+, \quad b_k \leq \left(\tilde{N}_k - q_k\right)_+. \tag{40}$$

同时满足能量平衡、功率和库存边界，并以真实库存为初值。该模型只以常规富余电充电，紧急电只补缺口，午夜库存价值取 0；每次只执行第一段动作。只有到允许的发布时间，才更新完整预测并重申报常规计划。

两种反馈共用参数、预报发布时间、1 月热启动及正式期初库存，后续计划随各自真实状态更新。表63比较常规费、紧急费与总费，表64报告费用增量和边界库存；增费定义为滚动反馈总费减去原反馈总费。

表 63 确定性价格感知反馈与原反馈的费用比较（2—12 月，元）

方案	反馈规则	常规购电费	紧急购电费	总费用
问题 2	贪心	13,291,625.72	616,550.23	13,908,175.95
问题 2	确定性滚动	13,287,773.99	654,732.22	13,942,506.21
问题 3	贪心	13,177,405.10	138,430.89	13,315,835.99
问题 3	确定性滚动	13,062,906.94	466,563.01	13,529,469.95
问题 4-2	贪心	14,032,328.84	643,516.03	14,675,844.87
问题 4-2	确定性滚动	14,028,077.93	681,762.73	14,709,840.66
问题 4-3	贪心	13,917,214.62	127,670.62	14,044,885.25
问题 4-3	确定性滚动	13,807,269.57	451,105.26	14,258,374.82

表 64 滚动反馈的费用增量与物理统计

方案	增费（元）	紧急电（kWh）	未利用电（kWh）	期初 SOC（kWh）	期末 SOC（kWh）
问题 2	34,330.27	147,630.86	2,692,215.80	6,840.33	6,258.37
问题 3	213,633.96	128,007.58	2,053,973.08	7,089.87	5,987.72
问题 4-2	33,995.79	143,395.60	2,674,373.02	6,864.27	6,263.02
问题 4-3	213,489.58	120,113.54	2,040,532.00	7,114.54	5,997.11

四组滚动反馈均降低常规费，却增加了更多紧急费；期末库存分别与原反馈一致，因而费用差不能归于不同的边界库存。本对照没有重新校准参数或设置跨日库存价值，只能说明该配置未改善累计费用，不能证明原反馈最优。

B.4 上月选参的滚动评价

在 2025 年 4—12 月 275 天另作时序实验：每月开始前，仅用上一个完整月的费用在 30 个候选中重选融合权重与分位；历史光伏基准只在 $\lambda = 0$ 下选分位。候选评分均从同一参考月初库存开始，不添加末端库存估值，也不将候选月底库存传入正式路径。

固定参数、逐月融合、逐月历史三条正式路径共用 4 月 1 日库存，之后独立连续运行。累计账单与首末库存见表65，其 4—12 月费用不能直接与 2—12 月主结果相减。

表 65 按月选参与固定参数比较（2025 年 4—12 月）

方案	参数规则	总费用（元）	紧急电（kWh）	期初 SOC（kWh）	期末 SOC（kWh）
问题 3	固定 1 月参数	11,025,848.81	24,392.62	6,461.68	5,987.72
问题 3	上月选择融合	11,027,957.95	39,352.86	6,461.68	5,987.72
问题 3	上月选择纯历史	11,132,033.28	37,016.85	6,461.68	5,987.72
问题 4-3	固定 1 月参数	11,695,213.43	23,070.56	6,517.67	5,997.11
问题 4-3	上月选择融合	11,711,534.09	39,935.96	6,517.67	5,997.11
问题 4-3	上月选择纯历史	11,812,014.14	46,519.39	6,517.67	5,997.11

问题三和 4-3 的逐月融合方案比固定参数分别多支出 2109.14 元、16320.66 元，9 个月中仅 3 个月、1 个月更省；相对逐月历史基准则分别节省 104075.33 元、100480.05 元。逐月参数及每月差额保存在支撑文件，正文保留决定结论的累计比较。同年滚动评价仍属于再分析，不能据月度胜率推出统计显著性或未见年度泛化。

C 计算程序源码

C.1 数据预处理程序

```
1  """Validate source workbooks and expose explicit, reproducible time conventions.
2
3  Main arrays use right-endpoint power as the representative 10-minute power.
4  The *_trapezoid arrays provide the alternative piecewise-linear integral.
5  No fitted predictor, future-observation imputation, or outlier deletion is used.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 from datetime import date, datetime, time, timedelta
12 import hashlib
13 import json
14 from pathlib import Path
15
16 import numpy as np
17 import openpyxl
18
19
20 ROOT = Path(__file__).resolve().parents[1]
21 EXPECTED_DATES = [date(2025, 1, 1) + timedelta(days=i) for i in range(365)]
22 RELEASE_HOURS = (0, 6, 12, 18)
23
24
25 def minutes(value: object) -> int:
26     """Convert mixed Excel time cells, preserving the explicit next-day mark."""
27     if isinstance(value, datetime):
28         value = value.time()
29     if isinstance(value, time):
30         if value.second or value.microsecond:
31             raise ValueError(f"Unexpected fractional time: {value!r}")
32         return value.hour * 60 + value.minute
33     if isinstance(value, str):
34         text = value.strip()
35         day_offset = 1440 if text.endswith("+1") else 0
36         if day_offset:
37             text = text[:-2]
38         hour, minute = map(int, text.split(":"))
39         return day_offset + hour * 60 + minute
40     raise ValueError(f"Unsupported time cell: {value!r}")
41
42
43 def calendar_date(value: object) -> date:
```

```

44     if isinstance(value, datetime):
45         return value.date()
46     if isinstance(value, date):
47         return value
48     if isinstance(value, str):
49         return date(*map(int, value.split("-")))
50     raise ValueError(f"Unsupported date: {value!r}")
51
52
53 def checked_numeric(rows: list, shape: tuple[int, ...], label: str) -> np.ndarray:
54     # float(None) fails rather than silently treating a blank as zero.
55     out = np.asarray([[float(v) for v in row] for row in rows], dtype=np.float64)
56     if out.shape != shape:
57         raise ValueError(f"{label}: expected {shape}, received {out.shape}")
58     if not np.all(np.isfinite(out)):
59         raise ValueError(f"{label}: contains non-finite values")
60     if np.any(out < 0):
61         raise ValueError(f"{label}: contains negative values; inspect source")
62     return out
63
64
65 def read_daily(sheet, label: str) -> np.ndarray:
66     rows = list(sheet.values)
67     if [minutes(v) for v in rows[0][1:]] != list(range(10, 1441, 10)):
68         raise ValueError(f"{label}: time columns must run from 00:10 through 24:00")
69     if [calendar_date(row[0]) for row in rows[1:]] != EXPECTED_DATES:
70         raise ValueError(f"{label}: dates must be every day of 2025 in order")
71     return checked_numeric([row[1:] for row in rows[1:]], (365, 144), label)
72
73
74 def trapezoid_daily(values: np.ndarray) -> np.ndarray:
75     """Interval means for a continuous annual piecewise-linear power curve.
76
77     The one missing annual origin is held equal to the first sample. Subsequent
78     days start at the immediately preceding day's 24:00 observation.
79     """
80     flat = values.ravel()
81     left = np.concatenate([flat[0], flat[:-1]])
82     return ((left + flat) / 2).reshape(values.shape)
83
84
85 def summarize(values: np.ndarray) -> dict:
86     return {
87         "shape": list(values.shape),
88         "minimum": float(np.min(values)),
89         "maximum": float(np.max(values)),
90         "mean": float(np.mean(values)),
91         "missing_or_nonfinite": int(np.count_nonzero(~np.isfinite(values))),

```

```

92         "negative_count": int(np.count_nonzero(values < 0)),
93     }
94
95
96 def prepare(raw: Path, output: Path) -> dict:
97     paths = {i: raw / f"附件{i}.xlsx" for i in range(1, 5)}
98     with paths[1].open("rb") as stream:
99         wb = openpyxl.load_workbook(stream, read_only=True, data_only=True)
100         try:
101             rows = list(wb["Sheet1"].values)
102         finally:
103             wb.close()
104     if [minutes(row[0]) for row in rows[1:]] != list(range(10, 1441, 10)):
105         raise ValueError("Attachment 1: invalid time sequence")
106     q1 = checked_numeric([row[1:] for row in rows[1:]], (144, 3), "Attachment 1")
107
108     wb = openpyxl.load_workbook(paths[2], read_only=True, data_only=True)
109     try:
110         load = read_daily(wb["小区负载"], "Load")
111         pv = read_daily(wb["光伏发电实际功率"], "Actual PV")
112     finally:
113         wb.close()
114     wb = openpyxl.load_workbook(paths[4], read_only=True, data_only=True)
115     try:
116         price = read_daily(wb["Sheet1"], "Dynamic price")
117     finally:
118         wb.close()
119     wb = openpyxl.load_workbook(paths[3], read_only=True, data_only=True)
120     try:
121         forecast_rows = list(wb["Sheet1"].values)
122     finally:
123         wb.close()
124     if list(forecast_rows[0][2:]) != [f"预报{i}小时" for i in range(1, 25)]:
125         raise ValueError("Attachment 3: forecast horizons are not hours +1 through +24")
126     if len(forecast_rows) != 1461:
127         raise ValueError("Attachment 3: expected 1460 issue rows")
128     inherited = None
129     for i, row in enumerate(forecast_rows[1:]):
130         if row[0] not in (None, ""):
131             inherited = calendar_date(row[0])
132             expected_date = EXPECTED_DATES[i // 4]
133             expected_hour = RELEASE_HOURS[i % 4]
134             if inherited != expected_date or minutes(row[1]) != expected_hour * 60:
135                 raise ValueError(f"Attachment 3: incorrect issue date/hour at row {i + 2}")
136     forecast_hourly = checked_numeric(
137         [row[2:] for row in forecast_rows[1:]], (1460, 24), "PV forecasts"
138     ).reshape(365, 4, 24)
139

```

```

140 # The interpolation origin is available at issue time, not after it.
141 anchors = np.zeros((365, 4), dtype=np.float64)
142 anchors[1:, 0] = pv[:-1, -1]
143 for issue_index, hour in enumerate(RELEASE_HOURS[1:], start=1):
144     anchors[:, issue_index] = pv[:, hour * 6 - 1]
145 hourly_knots = np.concatenate((anchors[... , None], forecast_hourly), axis=2)
146 fine_knots = np.empty((365, 4, 145), dtype=np.float64)
147 for d in range(365):
148     for issue_index in range(4):
149         fine_knots[d, issue_index] = np.interp(
150             np.arange(145) / 6, np.arange(25), hourly_knots[d, issue_index]
151         )
152
153 arrays = {
154     "dates": np.array([d.isoformat() for d in EXPECTED_DATES]),
155     "load": load,
156     "pv": pv,
157     "fixed_price": q1[:, 0],
158     "dynamic_price": price,
159     "q1_load": q1[:, 1],
160     "q1_pv": q1[:, 2],
161     "forecast": fine_knots[:, :, 1:],
162     "load_trapezoid": trapezoid_daily(load),
163     "pv_trapezoid": trapezoid_daily(pv),
164     "q1_load_trapezoid": (np.roll(q1[:, 1], 1) + q1[:, 1]) / 2,
165     "q1_pv_trapezoid": (np.roll(q1[:, 2], 1) + q1[:, 2]) / 2,
166     "forecast_trapezoid": (fine_knots[:, :, :-1] + fine_knots[:, :, 1:]) / 2,
167     "forecast_hourly": forecast_hourly,
168     "forecast_anchor": anchors,
169     "forecast_release_hours": np.array(RELEASE_HOURS, dtype=np.int64),
170     "interval_end_minutes": np.arange(10, 1441, 10, dtype=np.int64),
171     "dt_hours": np.array(1 / 6),
172 }
173
174 # At exact hourly targets, the right-endpoint interpolant must reproduce
175 # the supplied forecasts; no day shift or averaging is allowed here.
176 np.testing.assert_allclose(arrays["forecast"][:, :, 5::6], forecast_hourly,
177                             rtol=0, atol=1e-10)
178
179 # Integration of the linear interpolant must preserve hourly trapezoids.
180 np.testing.assert_allclose(
181     arrays["forecast_trapezoid"].reshape(365, 4, 24, 6).mean(axis=3),
182     (hourly_knots[:, :, :-1] + hourly_knots[:, :, 1:]) / 2,
183     rtol=1e-13, atol=1e-10,
184 )
185
186 output.mkdir(parents=True, exist_ok=True)
187 np.savez_compressed(output / "data.npz", **arrays)
188
189 audit = {
190     "source_sha256": {p.name: hashlib.sha256(p.read_bytes()).hexdigest()
191                       for p in paths.values()}},

```

```

188     "date_start": "2025-01-01",
189     "date_end": "2025-12-31",
190     "days": 365,
191     "submission_start": "2025-02-01",
192     "submission_days": 334,
193     "interval_minutes": 10,
194     "main_power_convention": "right_endpoint_representative_interval_mean",
195     "forecast_origin": "PV observation at issue time; Jan 1 00:00 = 0 kW",
196     "statistics": {key: summarize(value) for key, value in arrays.items()
197                    if key != "dates"},
198     "annual_energy_kwh": {"load": float(load.sum() / 6),
199                          "pv": float(pv.sum() / 6),
200                          "load_trapezoid": float(arrays["load_trapezoid"].sum() / 6),
201                          "pv_trapezoid": float(arrays["pv_trapezoid"].sum() / 6)},
202     "q1_energy_kwh": {"load": float(q1[:, 1].sum() / 6),
203                      "pv": float(q1[:, 2].sum() / 6)},
204     "a1_versus_annual_same_time_average_max_abs_kw": {
205         "load": float(np.max(np.abs(load.mean(axis=0) - q1[:, 1]))),
206         "pv": float(np.max(np.abs(pv.mean(axis=0) - q1[:, 2]))),
207     },
208     "duplicate_daily_rows": {"load": 365 - len(np.unique(load, axis=0)),
209                             "pv": 365 - len(np.unique(pv, axis=0)),
210                             "dynamic_price": 365 - len(np.unique(price, axis=0))},
211     "versions": {"numpy": np.__version__, "openpyxl": openpyxl.__version__},
212 }
213 (output / "audit.json").write_text(json.dumps(audit, ensure_ascii=False, indent=2) + "\n",
214                                     encoding="utf-8")
215
216 return audit
217
218 def main() -> None:
219     parser = argparse.ArgumentParser(description=__doc__)
220     parser.add_argument("--raw", type=Path, default=ROOT / "data" / "raw")
221     parser.add_argument("--output", type=Path, default=ROOT / "data" / "processed")
222     args = parser.parse_args()
223     audit = prepare(args.raw, args.output)
224     print(json.dumps({"output": str(args.output / "data.npz"),
225                     "days": audit["days"],
226                     "annual_energy_kwh": audit["annual_energy_kwh"]},
227                   ensure_ascii=False, indent=2))
228
229
230 if __name__ == "__main__":
231     main()

```

C.2 四问共用计算模型

```
1  """Causal forecasts, risk-quantile linear planning and feasible battery feedback.
2
3  All power input is kW; optimization and execution variables are AC-side kWh.
4  """
5  from __future__ import annotations
6
7  from dataclasses import dataclass
8  from functools import lru_cache
9  import numpy as np
10 from scipy.optimize import linprog
11 from scipy.sparse import lil_matrix, csr_matrix, hstack
12
13 DT = 1 / 6
14 ETA = 0.9
15 LOW, HIGH, INITIAL = 1200.0, 10800.0, 6000.0
16 LIMIT = 5000 * DT
17
18
19 @lru_cache(maxsize=8)
20 def lp_structure(n, eta=ETA):
21     # x = [grid, charge, discharge, state_after_slot, unused_energy]
22     a = lil_matrix((2 * n, 5 * n))
23     for t in range(n):
24         a[t, t], a[t, n+t], a[t, 2*n+t], a[t, 4*n+t] = 1, -1, 1, -1
25         a[n+t, n+t], a[n+t, 2*n+t], a[n+t, 3*n+t] = -eta, 1/eta, 1
26         if t:
27             a[n+t, 3*n+t-1] = -1
28     return a.tocsr()
29
30
31 def optimize(net, price, soc, base=None, refund=False, terminal=INITIAL, tie_break=True,
32             terminal_equal=False, eta=ETA, settlement='final_net'):
33     """Deterministic LP for the remaining delivery slots.
34
35     ``base`` is the midnight nomination under final-net settlement, or the last
36     committed nomination under per-revision settlement. Previously incurred
37     costs are sunk. With positive prices, free disposal and no refund, lowering
38     that base is dominated by keeping the commitment and discarding its energy.
39
40     if settlement not in ('final_net', 'per_revision'):
41         raise ValueError(f'Unknown settlement: {settlement}')
42     if settlement == 'per_revision' and refund:
43         raise ValueError('refund=True is only supported with final_net settlement')
44     net, price = np.asarray(net, float), np.asarray(price, float)
45     n = len(net)
```

```

46     a = lp_structure(n,eta)
47     b = np.r_[net, soc, np.zeros(n-1)]
48     cost = np.zeros(5*n)
49     cost[:n] = price if base is None else 1.5 * price
50     if tie_break:
51         cost[n:3*n] = 1e-6
52         cost[4*n:] = 1e-7
53     lower = np.zeros(5*n)
54     upper = np.full(5*n, np.inf)
55     lower[3*n:4*n], upper[3*n:4*n] = LOW, HIGH
56     upper[n:3*n] = LIMIT
57     lower[4*n-1] = terminal
58     if terminal_equal:
59         upper[4*n-1] = terminal
60     if base is not None and not refund:
61         lower[:n] = base
62     aub, bub = None, None
63     if base is not None and refund:
64         # q=base+up-down; cost=1.5*p*up-0.5*p*down.
65         m = lil_matrix((n, 7*n))
66         for t in range(n):
67             m[t,t], m[t,5*n+t], m[t,6*n+t] = 1, -1, 1
68         from scipy.sparse import vstack
69         a = vstack([hstack([a, csr_matrix((2*n,2*n))]), m.tocsr()]).tocsr()
70         b = np.r_[b, base]
71         cost[:n] = 0
72         cost = np.r_[cost, 1.5*price, -0.5*price]
73         lower = np.r_[lower, np.zeros(2*n)]
74         upper = np.r_[upper, np.full(n,np.inf), base]
75     result = linprog(cost, A_eq=a, b_eq=b, bounds=np.c_[lower,upper], method='highs')
76     if not result.success:
77         raise RuntimeError(f'LP failed: {result.message}; n={n}, soc={soc}')
78     x = result.x
79     return dict(grid=x[:n], charge=x[n:2*n], discharge=x[2*n:3*n],
80                soc=np.r_[soc,x[3*n:4*n]], spill=x[4*n:5*n],
81                objective=float(result.fun), primal_residual=float(np.max(np.abs(a@x-b))))
82
83
84 def q1_solution(data, trapezoid=False, eta=ETA):
85     suffix = '_trapezoid' if trapezoid else ''
86     net = (data['q1_load'+suffix]-data['q1_pv'+suffix])*DT
87     sol = optimize(net, data['fixed_price'], INITIAL, terminal_equal=True,eta=eta)
88     exact = optimize(net, data['fixed_price'], INITIAL, tie_break=False, terminal_equal=
True,eta=eta)
89     sol['cost'] = float(sol['grid']@data['fixed_price'])
90     sol['unperturbed_lower_bound'] = float(exact['grid']@data['fixed_price'])
91     sol['optimality_gap_yuan'] = sol['cost']-sol['unperturbed_lower_bound']
92     sol['net'] = net

```

```

93     return sol
94
95
96 def weighted_profile(history, day, weekly=True):
97     """Recent same-weekday profiles. No actual from day or later is read."""
98     if day == 0:
99         # Explicit cold-start prior; only January initialization is affected.
100         return np.full(144, 4500.0) if weekly else np.zeros(144)
101     if weekly and day >= 7:
102         idx = np.arange(day-7, max(-1,day-29), -7)
103         weights = 0.55 ** np.arange(len(idx))
104     else:
105         idx = np.arange(day-1, max(-1,day-8), -1)
106         weights = 0.8 ** np.arange(len(idx))
107     return np.average(history[idx], axis=0, weights=weights)
108
109
110 def historical_pv(history, day):
111     if day == 0:
112         return np.zeros(144)
113     idx = np.arange(day-1, max(-1,day-8), -1)
114     return np.average(history[idx], axis=0, weights=0.75*np.arange(len(idx)))
115
116
117 def published_pv(data, day, issue, forecast_sources=None, anchor_updates=True):
118     """Available hourly forecast, aligned to absolute targets with a causal anchor.
119
120     forecast_sources[r] is the latest allowed release row at decision r.
121     The default route preserves the original preprocessed forecast exactly.
122     """
123     sources = tuple(range(4)) if forecast_sources is None else tuple(forecast_sources)
124     if len(sources) != 4 or any(not isinstance(s, (int, np.integer)) or s < 0 or s > r
125                                for r,s in enumerate(sources)):
126         raise ValueError('forecast_sources must contain four release indices available by
each issue')
127     start = issue*36
128     source = sources[issue]
129     if source == issue and (anchor_updates or issue == 0):
130         return data['forecast'][day,issue,:144-start].copy()
131     offset = (issue-source)*6
132     horizon_hours = (144-start)//6
133     if anchor_updates:
134         anchor = data['forecast_anchor'][day,issue]
135     elif offset:
136         anchor = data['forecast_hourly'][day,source,offset-1]
137     else:
138         previous_source = sources[issue-1]
139         anchor = data['forecast_hourly'][day,previous_source,(issue-previous_source)*6-1]

```

```

140     knots = np.r_[anchor,data['forecast_hourly']][day,source,offset:offset+horizon_hours]]
141     return np.interp(np.arange(1,145-start)/6,np.arange(len(knots)),knots)
142
143
144 def point_forecast(data, day, issue, use_forecast, dynamic=False, oracle_price=False,
145                   pv_weight=None, forecast_sources=None, load_update=True, anchor_updates=
146                   True):
147     start = issue*36
148     load = weighted_profile(data['load'], day)
149     if start and load_update:
150         # Issue-time load update uses only completed 10-minute slots.
151         recent = slice(max(0,start-18),start)
152         ratio = np.mean(data['load'][day,recent])/max(np.mean(load[recent]),1)
153         load = load*np.clip(ratio,0.75,1.25)
154     weight = float(use_forecast) if pv_weight is None else float(pv_weight)
155     if not 0 <= weight <= 1:
156         raise ValueError('pv_weight must be between zero and one')
157     if weight == 0:
158         pv = historical_pv(data['pv'],day)[start:]
159     else:
160         pv = published_pv(data,day,issue,forecast_sources,anchor_updates)
161         if weight != 1:
162             pv = weight*pv+(1-weight)*historical_pv(data['pv'],day)[start:]
163     if not dynamic:
164         price = data['fixed_price'][start:]
165     elif oracle_price:
166         price = data['dynamic_price'][day,start:]
167     elif day == 0:
168         price = data['fixed_price'][start:]
169     else:
170         price = weighted_profile(data['dynamic_price'],day)
171         if start:
172             recent = slice(max(0,start-18),start)
173             ratio = np.mean(data['dynamic_price'][day,recent])/max(np.mean(price[recent]),
174 ,0.01)
175             price = price*np.clip(ratio,0.5,1.5)
176             price = np.maximum(price[start:],0.001)
177     return (load[start:]-pv)*DT, np.asarray(price), load[start:], pv
178
179 @dataclass
180 class ForecastCache:
181     net: np.ndarray
182     price: np.ndarray
183     errors: np.ndarray
184     load: np.ndarray
185     pv: np.ndarray

```

```

186
187 def build_cache(data, forecast=False, dynamic=False, oracle_price=False,
188                 pv_weight=None, forecast_sources=None, load_update=True, anchor_updates=
    True):
189     shape = (365,4,144)
190     net, price, errors, load, pv = [np.full(shape,np.nan) for _ in range(5)]
191     actual = (data['load']-data['pv'])*DT
192     for day in range(365):
193         for issue in range(4):
194             t = issue*36
195             z,p,l,v = point_forecast(data,day,issue,forecast,dynamic,oracle_price,
196                                     pv_weight,forecast_sources,load_update,anchor_updates)
197             net[day,issue,t:],price[day,issue,t:] = z,p
198             load[day,issue,t:],pv[day,issue,t:] = l,v
199             errors[day,issue,t:] = actual[day,t:]-z
200     return ForecastCache(net,price,errors,load,pv)
201
202
203 def risk_forecast(cache, day, issue, quantile):
204     start = issue*36
205     net = cache.net[day,issue,start:].copy()
206     # Stored future errors are excluded explicitly. First 7 days are warmup only.
207     lo = max(7,day-28)
208     if day > lo:
209         past_errors = cache.errors[lo:day,issue,start:]
210         # Pool 7 neighboring delivery slots to stabilize small-sample quantiles.
211         pad = np.pad(past_errors,((0,0),(3,3)),mode='edge')
212         samples = np.concatenate([pad[:,j:j+len(net)] for j in range(7)],axis=0)
213         net += np.quantile(samples,quantile,axis=0)
214     return net
215
216
217 def execute_slot(grid, net, soc):
218     surplus = grid-net
219     if surplus >= 0:
220         charge = min(surplus,LIMIT,max(0,(HIGH-soc)/ETA))
221         return charge,0.0,soc+ETA*charge,0.0,surplus-charge
222     discharge = min(-surplus,LIMIT,max(0,(soc-LOW)*ETA))
223     return 0.0,discharge,soc-discharge/ETA,-surplus-discharge,0.0
224
225
226 def settle_revisions(plan, revisions, price, settlement='final_net', refund=False):
227     """Recompute adjustment charges from a frozen path of nominations.
228
229     ``plan`` and ``price`` have shape (... , slots); price may broadcast to plan.
230     ``revisions`` has shape (... , issues, slots): layer 0 is the original plan,
231     and NaN in later layers means that delivery slot was not revised. The
232     returned revision_up/down record every actual change relative to the

```

```

233 preceding commitment, even when final_net only bills the final net change.
234 Original-plan and emergency charges are excluded from adjustment_cost.
235 """
236 if settlement not in ('final_net', 'per_revision'):
237     raise ValueError(f'Unknown settlement: {settlement}')
238 if settlement == 'per_revision' and refund:
239     raise ValueError('refund=True is only supported with final_net settlement')
240 plan, revisions, price = (np.asarray(x, float) for x in (plan, revisions, price))
241 if (revisions.ndim != plan.ndim + 1 or revisions.shape[:-2] != plan.shape[:-1]
242     or revisions.shape[-1] != plan.shape[-1] or revisions.shape[-2] < 1):
243     raise ValueError('revisions must have shape (... , issues, slots) matching plan')
244 first = revisions[..., 0, :]
245 if np.any(np.isfinite(first) & ~np.isclose(first, plan, rtol=0, atol=1e-7)):
246     raise ValueError('revisions layer 0 must match the original plan')
247 current = plan.copy()
248 revision_up, revision_down = np.zeros_like(revisions), np.zeros_like(revisions)
249 for issue in range(1, revisions.shape[-2]):
250     proposed = revisions[..., issue, :]
251     next_plan = np.where(np.isfinite(proposed), proposed, current)
252     delta = next_plan - current
253     revision_up[..., issue, :] = np.maximum(delta, 0)
254     revision_down[..., issue, :] = np.maximum(-delta, 0)
255     current = next_plan
256 if settlement == 'per_revision':
257     up, down = revision_up.sum(axis=-2), revision_down.sum(axis=-2)
258 else:
259     up, down = np.maximum(current-plan, 0), np.maximum(plan-current, 0)
260 adjustment_cost = np.sum(price*(1.5*up+(-.5 if refund else .5)*down), axis=-1)
261 return dict(final=current, revision_up=revision_up, revision_down=revision_down,
262            up=up, down=down, adjustment_cost=adjustment_cost)
263
264
265 def simulate(data, cache, quantile, issues=(), refund=False, days=365, initial=INITIAL,
266            dynamic=False, progress=False, warmup_quantile=0.8, terminal=INITIAL,
267            settlement='final_net', warmup_cache=None):
268     if settlement not in ('final_net', 'per_revision'):
269         raise ValueError(f'Unknown settlement: {settlement}')
270     if settlement == 'per_revision' and refund:
271         raise ValueError('refund=True is only supported with final_net settlement')
272     shape = (days,144)
273     plan, adjusted, charge, discharge, emergency, spill = [np.zeros(shape) for _ in range
274     (6)]
275     state = np.zeros((days,145))
276     revisions = np.full((days,4,144),np.nan)
277     revision_up, revision_down = np.zeros_like(revisions), np.zeros_like(revisions)
278     daily_costs = np.zeros((days,4)) # original plan, adjustment, emergency, total
279     soc = initial
280     for day in range(days):

```

```

280     active_cache = warmup_cache if day < 31 and warmup_cache is not None else cache
281     active_quantile = warmup_quantile if day < 31 and warmup_quantile is not None else
quantile
282     base_net = risk_forecast(active_cache,day,0,active_quantile)
283     sol = optimize(base_net,active_cache.price[day,0],soc,terminal=terminal)
284     plan[day] = sol['grid']
285     adjusted[day] = plan[day]
286     revisions[day,0] = plan[day]
287     state[day,0] = soc
288     for t in range(144):
289         if t in issues:
290             issue = t//36
291             net = risk_forecast(active_cache,day,issue,active_quantile)
292             base = adjusted[day,t:] if settlement == 'per_revision' else plan[day,t:]
293             sol = optimize(net,active_cache.price[day,issue,t:],soc,base=base,refund=
refund,
294                             terminal=terminal,settlement=settlement)
295             adjusted[day,t:] = sol['grid']
296             revisions[day,issue,t:] = sol['grid']
297             net_actual = (data['load'][day,t]-data['pv'][day,t])*DT
298             c,d,soc,e,w = execute_slot(adjusted[day,t],net_actual,soc)
299             charge[day,t],discharge[day,t],emergency[day,t],spill[day,t] = c,d,e,w
300             state[day,t+1] = soc
301             price = data['dynamic_price'][day] if dynamic else data['fixed_price']
302             billed = settle_revisions(plan[day],revisions[day],price,settlement=settlement,
refund=refund)
303             revision_up[day], revision_down[day] = billed['revision_up'], billed['revision_down
']
304             basecost = price@plan[day]
305             adjcost = billed['adjustment_cost']
306             emcost = 5*price@emergency[day]
307             daily_costs[day] = [basecost,adjcost,emcost,basecost+adjcost+emcost]
308             if progress and (day+1)%60 == 0:
309                 print(f' day {day+1}: cumulative cost {daily_costs[:day+1,3].sum():.0f}',
flush=True)
310             return dict(plan=plan,adjusted=adjusted,charge=charge,discharge=discharge,
311                         emergency=emergency,spill=spill,soc=state,revisions=revisions,
312                         revision_up=revision_up,revision_down=revision_down, costs=daily_costs)
313
314
315 def summarize(result, start=31, end=None):
316     sl = slice(start,end)
317     costs = result['costs'][sl].sum(0)
318     return dict(plan_kwh=float(result['plan'][sl].sum()),
319                 adjusted_kwh=float(result['adjusted'][sl].sum()),
320                 emergency_kwh=float(result['emergency'][sl].sum()),
321                 spill_kwh=float(result['spill'][sl].sum()),
322                 charge_kwh=float(result['charge'][sl].sum()),

```

```
323         discharge_kwh=float(result['discharge'][s1].sum()),
324         plan_cost=float(costs[0]),adjustment_cost=float(costs[1]),
325         emergency_cost=float(costs[2]),total_cost=float(costs[3]),
326         start_soc=float(result['soc'][start,0]),end_soc=float(result['soc'][s1
    ] [-1,-1]),
327         emergency_days=int(np.sum(np.any(result['emergency'][s1]>1e-7,axis=1))))
```

C.3 主求解与结果归档程序

```
1  """Run the four questions, January-only calibration, ablations and sensitivity."""
2  from __future__ import annotations
3  import json
4  from pathlib import Path
5  import time
6  import hashlib
7  import itertools
8  import numpy as np
9  import scipy
10 from model import (DT, INITIAL, build_cache, q1_solution, simulate, summarize,
11                    settle_revisions)
12
13 ROOT = Path(__file__).resolve().parents[1]
14 OUT = ROOT/'outputs'
15 QUANTILES = (0.5,0.65,0.7,0.8,0.9,0.95)
16 PV_WEIGHTS = (0.0,0.25,0.5,0.75,1.0)
17
18 def plain(obj):
19     if isinstance(obj,np.ndarray):
20         return obj.tolist()
21     if isinstance(obj,np.generic):
22         return obj.item()
23     raise TypeError(type(obj).__name__)
24
25
26 def workbook_payload(data, q1, results):
27     payload = dict(schemaVersion=1,slotConvention='right-end-0000-to-2400',
28                    q1=dict(plan=q1['grid'],charge=q1['charge'],discharge=q1['discharge'],
29                           socStart=q1['soc'][0],socEnd=q1['soc'][-1]))
30     for name,result in results.items():
31         days = []
32         for d in range(31,365):
33             row = dict(date=str(data['dates'][d]),plan=result['plan'][d],
34                       planCost=result['costs'][d,0],charge=result['charge'][d],
35                       discharge=result['discharge'][d],socStart=result['soc'][d,0],
36                       socEnd=result['soc'][d,-1],emergency=result['emergency'][d],
37                       emergencyCost=result['costs'][d,2],totalCost=result['costs'][d,3])
38             if name in ('q3','q4_3'):
39                 row.update(adjusted=result['adjusted'][d],
40                           adjustedCost=result['costs'][d,0]+result['costs'][d,1])
41             days.append(row)
42         payload[name] = dict(days=days)
43     return payload
44
```

```

45
46 def main():
47     started = time.perf_counter()
48     (OUT/'main').mkdir(parents=True, exist_ok=True)
49     (OUT/'experiments/revision').mkdir(parents=True, exist_ok=True)
50     data = dict(np.load(ROOT/'data/processed/data.npz'))
51     q1 = q1_solution(data)
52     trap = q1_solution(data, trapezoid=True)
53     roundtrip = q1_solution(data, eta=float(np.sqrt(.9)))
54     caches, selected, calibration, results, weights = {}, {}, {}, {}, {}
55     memo, archived = {}, {}
56     revision = dict(schema_version=1, selection={}, scenes={}, baselines={},
57                     information_ablation={}, contracts={},
58                     validation_days=[14, 31], submission_days=[31, 365],
59                     candidate_family_post_review=True,
60                     warmup=dict(quantile=0.8, pv_weight=1.0, ends_before_day=31,
61                                 applies_to='scenes with warmup_mode=attachment3_fixed'))
62
63     def cache_for(dynamic=False, weight=1.0, sources=(0, 1, 2, 3), load_update=True,
64                  anchor_updates=True):
65         key=(dynamic, weight, tuple(sources), load_update, anchor_updates)
66         if key not in memo:
67             memo[key]=build_cache(data, forecast=True, dynamic=dynamic, pv_weight=weight,
68                                   forecast_sources=sources, load_update=load_update,
69                                   anchor_updates=anchor_updates)
70         return memo[key]
71
72     def calibrate(dynamic, weight_candidates, issues=(36, 72, 108), settlement='final_net'):
73         trials=[]
74         for weight in weight_candidates:
75             for tau in QUANTILES:
76                 r=simulate(data, cache_for(dynamic, weight), tau, issues=issues, days=31,
77                             dynamic=dynamic, warmup_quantile=None, settlement=settlement)
78                 score=summarize(r, start=14)
79                 trials.append(dict(pv_weight=weight, quantile=tau,
80                                   validation_cost=score['total_cost'],
81                                   validation_emergency_kwh=score['emergency_kwh']))
82         choice=min(trials, key=lambda x:(x['validation_cost'], x['pv_weight'], x['quantile']))
83         return dict(selected=choice, candidates=trials)
84
85     def scene(scene_id, label, weight, tau, dynamic=False, issues=(36, 72, 108),
86              sources=(0, 1, 2, 3), load_update=True, anchor_updates=True,
87              settlement='final_net', warmup_mode='self'):
88         print(f'Scene {scene_id}: w={weight}, tau={tau}, {settlement}', flush=True)
89         cache=cache_for(dynamic, weight, sources, load_update, anchor_updates)
90         warm=cache_for(dynamic, 1.0) if warmup_mode=='attachment3_fixed' else None
91         r=simulate(data, cache, tau, issues=issues, dynamic=dynamic, settlement=settlement,
92                    warmup_cache=warm)

```

```

92     archived[scene_id]=r
93     revision['scenes'][scene_id]=dict(label=label,pv_weight=weight,quantile=tau,
94         issues=list(issues),dynamic=dynamic,settlement=settlement,
95         forecast_sources=list(sources),load_update=load_update,anchor_updates=
anchor_updates,
96         warmup_mode=warmup_mode,soc_observation='own_actual',summary=summarize(r))
97     return r
98
99     # Keep Q2/Q4-2 unchanged. New PV-model candidates are selected on January only.
100     for name,dynamic in [('q2',False),('q4_2',True)]:
101         print(f'January calibration: {name}',flush=True)
102         fit=calibrate(dynamic,(0.0,),issues=())
103         tau=fit['selected']['quantile']
104         selected[name],calibration[name],weights[name]=tau,fit['candidates'],0.0
105         caches[name]=cache_for(dynamic,0.0)
106         results[name]=simulate(data,caches[name],tau,dynamic=dynamic,progress=True)
107
108     for name,dynamic in [('q3',False),('q4_3',True)]:
109         print(f'January joint PV/quantile calibration: {name}',flush=True)
110         fit=calibrate(dynamic,PV_WEIGHTS)
111         revision['selection'][name]=fit
112         choice=fit['selected']; tau,weight=choice['quantile'],choice['pv_weight']
113         selected[name],weights[name]=tau,weight
114         calibration[name]=[x for x in fit['candidates'] if x['pv_weight']==weight]
115         caches[name]=cache_for(dynamic,weight)
116         revision['baselines'][name]={}
117         for w,label,kind in [(1.0,'原纯附件3基准','pure_attachment3'),(0.0,'历史光伏基准','
historical')]:
118             baseline_choice=min((x for x in fit['candidates'] if x['pv_weight']==w),key=
lambda x:x['validation_cost'])
119             scene_id=name+('_legacy' if w==1 else '_historical')
120             scene(scene_id,label,w,baseline_choice['quantile'],dynamic=dynamic)
121             revision['scenes'][scene_id]['validation_cost']=baseline_choice['
validation_cost']
122             revision['baselines'][name][kind]=scene_id
123             if w == 0:
124                 common_id=name+'_historical_fixed'
125                 scene(common_id,'历史光伏基准（共同热启动）',w,baseline_choice['quantile'],
126                     dynamic=dynamic,warmup_mode='attachment3_fixed')
127                 revision['scenes'][common_id]['validation_cost']=baseline_choice['
validation_cost']
128                 revision['baselines'][name]['historical_common_warmup']=common_id
129                 results[name]=scene(name+'_selected','修订主策略',weight,tau,dynamic=dynamic,
130                     warmup_mode='attachment3_fixed')
131                 print(name,'selected',choice,'summary',summarize(results[name]),flush=True)
132
133     summary = dict(parameters=selected,calibration=calibration,
134         pv_weights=weights,revision_experiments='../experiments/revision/

```

```

experiments.json',
135         q1={k:v for k,v in q1.items() if np.isscalar(v)},
136         q1_grid_kwh=float(q1['grid'].sum()),
137         q1_trapezoid_cost=trap['cost'],q1_trapezoid_grid_kwh=float(trap['grid'].
sum()),
138         q1_roundtrip90_cost=roundtrip['cost'],
139         primary={k:summarize(v) for k,v in results.items()},
140         sensitivity={},forecast_accuracy={},versions=dict(numpy=np.__version__,
scipy=scipy.__version__))
141     for name,cache in caches.items():
142         err = cache.errors[31:,0]
143         summary['forecast_accuracy'][name] = dict(
144             net_mae_kw=float(np.mean(np.abs(err))/DT),
145             load_mae_kw=float(np.mean(np.abs(cache.load[31:,0]-data['load'][31:])),
146             pv_mae_kw=float(np.mean(np.abs(cache.pv[31:,0]-data['pv'][31:])),
147             price_mae=float(np.mean(np.abs(cache.price[31:,0]-(data['dynamic_price'][31:]
if '4' in name else data['fixed_price']))))))
148     # Controlled information experiments: current observations and update permissions
149     # remain available, only receipt of each new hourly forecast is switched.
150     for group,weight,tau,warmup in [('legacy',1.0,revision['scenes']['q3_legacy']['quantile
'], 'self'),
151                                     ('selected',weights['q3'],selected['q3'],'
attachment3_fixed'))]:
152         subset_ids=[]
153         for bits in itertools.product((0,1),repeat=3):
154             sources=[0]
155             for issue,enabled in enumerate(bits,1):
156                 sources.append(issue if enabled else sources[-1])
157             if all(bits):
158                 scene_id='q3_'+group
159             else:
160                 bitstr=''.join(map(str,bits)); scene_id=f'q3_{group}_info_{bitstr}'
161                 scene(scene_id,'新小时预报 '+bitstr,weight,tau,sources=tuple(sources),
warmup_mode=warmup)
162                 subset_ids.append(scene_id)
163                 revision['information_ablation'][group]=dict(hourly_subsets=subset_ids)
164
165     legacy_tau=revision['scenes']['q3_legacy']['quantile']
166     scene('q3_legacy_no_updates','仅00点计划, 无日内重规划',1.0,legacy_tau,issues=())
167     scene('q3_ladder_replan','开放重规划与实际SOC, 沿用日前负荷与PV',1.0,legacy_tau,
168           sources=(0,0,0,0),load_update=False,anchor_updates=False)
169     scene('q3_ladder_load','再增加已观测负荷修正',1.0,legacy_tau,
170           sources=(0,0,0,0),anchor_updates=False)
171     revision['information_ablation']['legacy']['ladder']=[
172         'q3_legacy_no_updates','q3_ladder_replan','q3_ladder_load','q3_legacy_info_000','
q3_legacy']
173
174     for name,dynamic,without in [('q3',False,'q2'),('q4_3',True,'q4_2')]:

```

```

175     contract={}
176     price=data['dynamic_price'] if dynamic else data['fixed_price']
177     for kind,scene_id in [('legacy',name+'_legacy'),('selected',name+'_selected')]:
178         frozen=archived[scene_id]
179         billed=settle_revisions(frozen['plan'],frozen['revisions'],price,settlement='
per_revision')
180         extra=float(billed['adjustment_cost'][31:].sum()-frozen['costs'][31:,1].sum())
181         frozen_total=summarize(frozen)['total_cost']+extra
182         contract['frozen_'+kind]=dict(scene_id=scene_id,extra_cost=extra,total_cost=
frozen_total,
183         withdrawn_kwh=float(billed['revision_down'][31:].sum()),
184         saving_vs_without_adjustment=summarize(results[without])['total_cost']-
frozen_total,
185         reoptimized=False)
186         print(f'Per-revision contract: January calibration {name}',flush=True)
187         fit=calibrate(dynamic,PV_WEIGHTS,settlement='per_revision')
188         choice=fit['selected']; scene_id=name+'_per_revision'
189         scene(scene_id,'逐笔合约重新优化',choice['pv_weight'],choice['quantile'],dynamic=
dynamic,
190         settlement='per_revision',warmup_mode='attachment3_fixed')
191         contract.update(selection=fit,reoptimized=scene_id)
192         revision['contracts'][name]=contract
193
194     # Same forecast/controller, varying only issued updates. Parameters remain frozen.
195     for name,issues in [('q3_midnight_only',()),('q3_update06',(36,)),
196         ('q3_update12',(72,)),('q3_update18',(108,)),
197         ('q3_update06_12',(36,72)),('q3_update06_18',(36,108)),
198         ('q3_update12_18',(72,108))]:
199         print(f'Ablation {name}',flush=True)
200         r = simulate(data,caches['q3'],selected['q3'],issues=issues,warmup_cache=cache_for(
False,1.0))
201         summary['sensitivity'][name] = summarize(r)
202     for key in ('q2','q3'):
203         for tau in (0.5,0.7,0.9):
204             print(f'Risk sensitivity {key} tau={tau}',flush=True)
205             issues = () if key=='q2' else (36,72,108)
206             r = simulate(data,caches[key],tau,issues=issues,
207                 warmup_cache=cache_for(False,1.0) if key=='q3' else None)
208             summary['sensitivity'][f'{key}_tau{tau}'] = summarize(r)
209     for key in ('q3','q4_3'):
210         print(f'Refund sensitivity {key}',flush=True)
211         r = simulate(data,caches[key],selected[key],issues=(36,72,108),refund=True,dynamic=
'4' in key,
212             warmup_cache=cache_for('4' in key,1.0))
213         summary['sensitivity'][f'{key}_refund'] = summarize(r)
214     for terminal in (3000.0,9000.0):
215         print(f'Terminal reserve sensitivity {terminal}',flush=True)
216         r = simulate(data,caches['q3'],selected['q3'],issues=(36,72,108),terminal=terminal,

```

```

217         warmup_cache=cache_for(False,1.0))
218     summary['sensitivity'][f'q3_terminal{int(terminal)}'] = summarize(r)
219     for key,forecasts,issues in [('q4_2',False,()),('q4_3',True,(36,72,108))]:
220         print(f'Perfect-price reference {key}',flush=True)
221         cache = build_cache(data,forecast=forecasts,dynamic=True,oracle_price=True,
pv_weight=weights[key])
222         warm=build_cache(data,forecast=True,dynamic=True,oracle_price=True) if forecasts
else None
223         r = simulate(data,cache,selected[key],issues=issues,dynamic=True,warmup_cache=warm)
224         summary['sensitivity'][f'{key}_known_price'] = summarize(r)
225     # Positive-price no-storage perfect-net reference: different information, no claim of
bound.
226     actual_net = np.maximum((data['load'][31:]-data['pv'][31:])*DT,0)
227     summary['ideal_no_storage_reference'] = dict(
228         fixed_cost=float(np.sum(actual_net*data['fixed_price'])),
229         dynamic_cost=float(np.sum(actual_net*data['dynamic_price'][31:])),
230         grid_kwh=float(actual_net.sum()))
231     summary['elapsed_seconds'] = time.perf_counter()-started
232     revision['source_sha256']={p:hashlib.sha256((ROOT/p).read_bytes()).hexdigest()
233         for p in ('scripts/model.py','scripts/solve.py','data/processed/data.npz','docs/
reviews/revision-plan.md')}
234     # Publish one coherent generation after every scenario is complete.
235     payload = workbook_payload(data,q1,results)
236     (OUT/'main/results.json').write_text(json.dumps(payload,default=plain,separators=(',',
:')),encoding='utf-8')
237     arrays = {f'q1_{k}':v for k,v in q1.items() if isinstance(v,np.ndarray)}
238     for name,result in results.items():
239         arrays.update({f'{name}_{k}':v for k,v in result.items()})
240     np.savez_compressed(OUT/'main/dispatch.npz',**arrays)
241     np.savez_compressed(OUT/'experiments/revision/dispatch.npz',**{
242         f'{name}_{k}':v for name,result in archived.items() for k,v in result.items()})
243     (OUT/'experiments/revision/experiments.json').write_text(json.dumps(revision,default=
plain,indent=2),encoding='utf-8')
244     (OUT/'main/summary.json').write_text(json.dumps(summary,default=plain,indent=2),
encoding='utf-8')
245     print(f'Completed in {summary["elapsed_seconds"]:.1f}s',flush=True)
246
247
248 if __name__ == '__main__':
249     main()

```
